

UNIT IV INTRODUCTION TO NUMPY AND PANDAS

9

Understanding Data Types in Python -The Basics of NumPy Arrays - Computation on NumPy Arrays: Universal Functions - Aggregations: Min, Max, and Everything In Between. Introducing Pandas Objects - Data Indexing and Selection - Operating on Data in Pandas -Handling Missing Data - Hierarchical Indexing - Combining Datasets: Concat and Append -Combining Datasets: Merge and Join - Aggregation and Grouping - Pivot Tables – Vectorized String Operations - Working with Time Series.

INTRODUCTION TO NUMPY

- Datasets can come from a wide range of sources and in a wide range of formats, including collections of documents, collections of images, collections of sound clips, collections of numerical measurements, or nearly anything else.
- Despite this apparent heterogeneity, many datasets can be represented fundamentally as arrays of numbers.
- For example,
 - images—particularly digital images—can be thought of as simply two-dimensional arrays of numbers representing pixel brightness across the area.
 - Sound clips can be thought of as one-dimensional arrays of intensity versus time.
 - Text can be converted in various ways into numerical representations, such as binary digits representing the frequency of certain words or pairs of words.
- No matter what the data is, the first step in making it analyzable will be to transform it into arrays of numbers
- For this reason, efficient storage and manipulation of numerical arrays is absolutely fundamental to the process of doing data science
- We'll now take a look at the specialized tools that Python has for handling such numerical arrays: The NumPy package and the Pandas package
- This part of the notes will cover NumPy in detail. NumPy (short for *Numerical Python*) provides an efficient interface to store and operate on dense data buffers.
- In some ways, NumPy arrays are like Python's built-in list type, but NumPy arrays provide much more efficient storage and data operations as the arrays grow larger in size.
- NumPy arrays form the core of nearly the entire ecosystem of data science tools in Python, so time spent learning to use NumPy effectively will be valuable no matter what aspect of data science interests you
- you can import NumPy and double-check the version:

```
import numpy
numpy.__version__
```
- you'll find that most people in the SciPy/PyData world will import NumPy using np as an alias:

```
import numpy as np
```
- To display all the contents of the NumPy namespace, you can type this:

```
np.<TAB>
```

- And to display NumPy's built-in documentation, you can use this:
`np?`
- More detailed documentation, along with tutorials and other resources, can be found at <http://www.numpy.org>

1. UNDERSTANDING DATA TYPES IN PYTHON

- Effective data-driven science and computation requires understanding how data is stored and manipulated
- Users of Python are often drawn in by its ease of use, one piece of which is dynamic typing. While a statically typed language like C or Java requires each variable to be explicitly declared, a dynamically typed language like Python skips this specification.
- For example, in C you might specify a particular operation as follows:

```
/* C code */  
int result = 0;  
for(int i=0; i<100; i++){  
    result += i;  
}
```

- While in Python the equivalent operation could be written this way:

```
# Python code  
result = 0  
for i in range(100):  
    result += i
```

- Notice the main difference: in C, the data types of each variable are explicitly declared, while in Python the types are dynamically inferred. This means, for example, that we can assign any kind of data to any variable

```
# Python code  
x = 4  
x = "four"
```

- Here we've switched the contents of x from an integer to a string. The same thing in C would lead (depending on compiler settings) to a compilation error or other unintended consequences

```
/* C code */  
int x = 4;  
x = "four"; // FAILS
```

- This sort of flexibility is one piece that makes Python and other dynamically typed languages convenient and easy to use.
- Understanding how this works is an important piece of learning to analyze data efficiently and effectively with Python.
- But what this type flexibility also points to is the fact that Python variables are more than just their value; they also contain extra information about the type of the value.

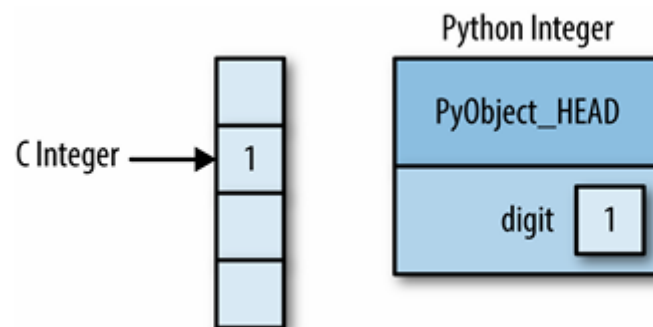
A Python Integer Is More Than Just an Integer

- The standard Python implementation is written in C. This means that every Python object is simply a cleverly disguised C structure, which contains not only its value, but other information as well.
- For example, when we define an integer in Python, such as `x = 10000`, `x` is not just a “raw” integer. It’s actually a pointer to a compound C structure, which contains several values.
- Looking through the Python 3.4 source code, we find that the integer (long) type definition effectively looks like this (once the C macros are expanded):

```

struct _longobject {
    long ob_refcnt;
    PyTypeObject *ob_type;
    size_t ob_size;
    long ob_digit[1];
};
  
```

- A single integer in Python 3.4 actually contains four pieces:
 - **ob_refcnt**, a reference count that helps Python silently handle memory allocation and deallocation.
 - **ob_type**, which encodes the type of the variable.
 - **ob_size**, which specifies the size of the following data members
 - **ob_digit**, which contains the actual integer value that we expect the Python variable to represent.
- This means that there is some overhead in storing an integer in Python as compared to an integer in a compiled language like C, as illustrated in F



- Here `PyObject_HEAD` is the part of the structure containing the reference count, type code, and other pieces mentioned before.
- Notice the difference here: a C integer is essentially a label for a position in memory whose bytes encode an integer value.
- A Python integer is a pointer to a position in memory containing all the Python object information, including the bytes that contain the integer value.
- This extra information in the Python integer structure is what allows Python to be coded so freely and dynamically.
- All this additional information in Python types comes at a cost, however, which becomes especially apparent in structures that combine many of these objects

A Python List Is More Than Just a List

- Let's consider now what happens when we use a Python data structure that holds many Python objects.
- The standard mutable multi element container in Python is the list.
- We can create a list of integers as follows:

```
In[1]: L = list(range(10))  
      L
```

```
Out[1]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In[2]: type(L[0])
```

```
Out[2]: int
```

Or, similarly, a list of strings:

```
In[3]: L2 = [str(c) for c in L]  
      L2
```

```
Out[3]: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
In[4]: type(L2[0])
```

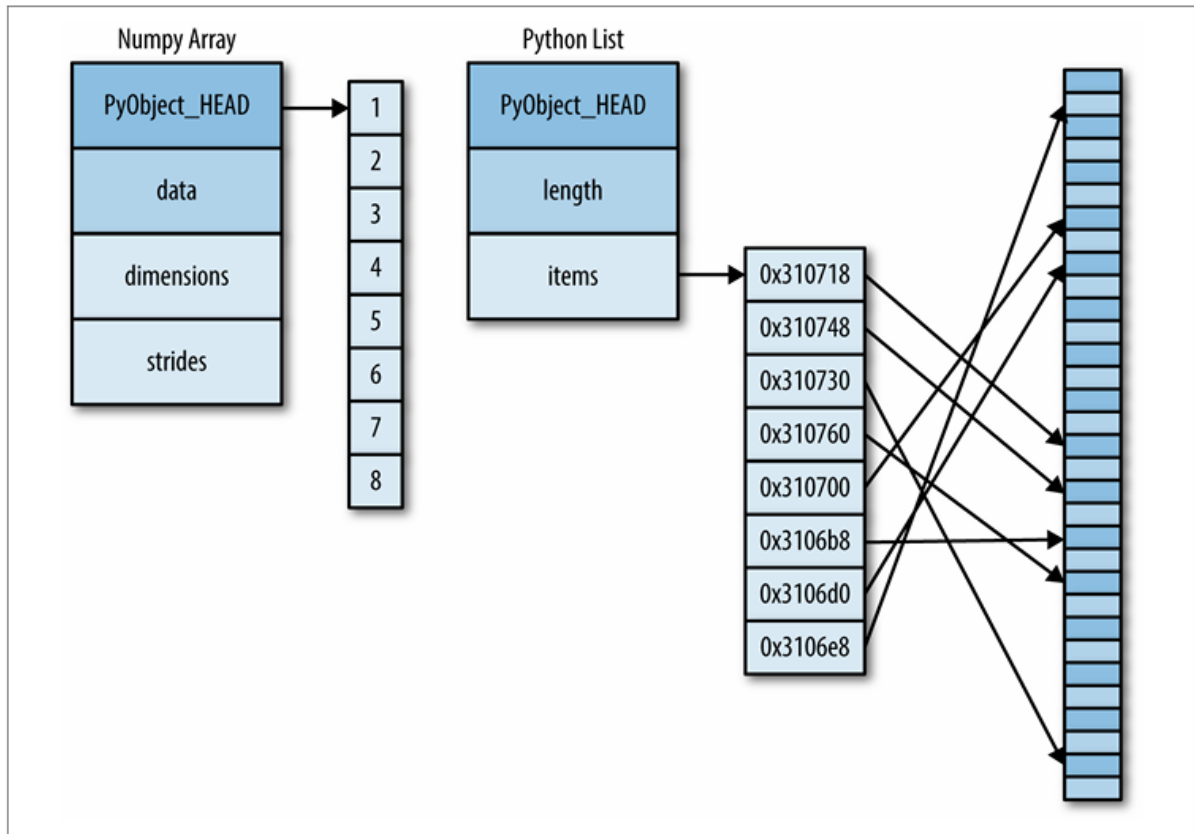
```
Out[4]: str
```

Because of Python's dynamic typing, we can even create heterogeneous lists:

```
In[5]: L3 = [True, "2", 3.0, 4]  
      [type(item) for item in L3]
```

```
Out[5]: [bool, str, float, int]
```

- But this flexibility comes at a cost: to allow these flexible types, each item in the list must contain its own type info, reference count, and other information—that is, each item is a complete Python object.
- In the special case that all variables are of the same type, much of this information is redundant: it can be much more efficient to store data in a fixed-type array.
- The difference between a dynamic-type list and a fixed-type (NumPy-style) array is illustrated in Figure.
- At the implementation level, the array essentially contains a single pointer to one contiguous block of data.
- The Python list, on the other hand, contains a pointer to a block of pointers, each of which in turn points to a full Python object like the Python integer we saw earlier.
- Again, the advantage of the list is flexibility: because each list element is a full structure containing both data and type information, the list can be filled with data of any desired type.
- Fixed-type NumPy-style arrays lack this flexibility, but are much more efficient for storing and manipulating data.



• Figure 2-2. The difference between C and Python lists

Fixed-Type Arrays in Python

- Python offers several different options for storing data in efficient, fixed-type data buffers.
- The built-in array module (available since Python 3.3) can be used to create dense arrays of a uniform type:

```
In[6]: import array
      L = list(range(10))
      A = array.array('i', L)
      A
```

```
Out[6]: array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Here 'i' is a type code indicating the contents are integers.

- Much more useful, however, is the ndarray object of the NumPy package.
- While Python's array object provides efficient storage of array-based data, NumPy adds to this efficient operations on that data.
- We will explore these operations in later sections; here we'll demonstrate several ways of creating a NumPy array.
- We'll start with the standard NumPy import, under the alias np:

```
In[7]: import numpy as np
```

Creating Arrays from Python Lists

- First, we can use `np.array` to create arrays from Python lists:

```
In[8]: # integer array:
       np.array([1, 4, 2, 5, 3])
```

```
Out[8]: array([1, 4, 2, 5, 3])
```

- Remember that unlike Python lists, NumPy is constrained to arrays that all contain the same type. If types do not match, NumPy will upcast if possible (here, integers are upcast to floating point):

```
In[9]: np.array([3.14, 4, 2, 3])
```

```
Out[9]: array([ 3.14,  4.  ,  2.  ,  3.  ])
```

- Finally, unlike Python lists, NumPy arrays can explicitly be multidimensional; here's one way of initializing a multidimensional array using a list of lists:

```
In[11]: # nested lists result in multidimensional arrays
        np.array([range(i, i + 3) for i in [2, 4, 6]])
```

```
Out[11]: array([[2, 3, 4],
                [4, 5, 6],
                [6, 7, 8]])
```

- The inner lists are treated as rows of the resulting two-dimensional array.

Creating Arrays from Scratch

- Especially for larger arrays, it is more efficient to create arrays from scratch using routines built into NumPy. Here are several examples:

```
In[12]: # Create a length-10 integer array filled with zeros
        np.zeros(10, dtype=int)
```

```
Out[12]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In[13]: # Create a 3x5 floating-point array filled with 1s
        np.ones((3, 5), dtype=float)
```

```
Out[13]: array([[ 1.,  1.,  1.,  1.,  1.],
                [ 1.,  1.,  1.,  1.,  1.],
                [ 1.,  1.,  1.,  1.,  1.]])
```

```
In[14]: # Create a 3x5 array filled with 3.14
        np.full((3, 5), 3.14)
```

```
Out[14]: array([[ 3.14,  3.14,  3.14,  3.14,  3.14],
                [ 3.14,  3.14,  3.14,  3.14,  3.14],
                [ 3.14,  3.14,  3.14,  3.14,  3.14]])
```

```
In[15]: # Create an array filled with a linear sequence
        # Starting at 0, ending at 20, stepping by 2
        # (this is similar to the built-in range() function)
        np.arange(0, 20, 2)
```

```
Out[15]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])

In[16]: # Create an array of five values evenly spaced between 0 and 1
        np.linspace(0, 1, 5)

Out[16]: array([ 0. ,  0.25,  0.5 ,  0.75,  1.  ])

In[17]: # Create a 3x3 array of uniformly distributed
        # random values between 0 and 1
        np.random.random((3, 3))

Out[17]: array([[ 0.99844933,  0.52183819,  0.22421193],
                [ 0.08007488,  0.45429293,  0.20941444],
                [ 0.14360941,  0.96910973,  0.946117  ]])

In[18]: # Create a 3x3 array of normally distributed random values
        # with mean 0 and standard deviation 1
        np.random.normal(0, 1, (3, 3))

Out[18]: array([[ 1.51772646,  0.39614948, -0.10634696],
                [ 0.25671348,  0.00732722,  0.37783601],
                [ 0.68446945,  0.15926039, -0.70744073]])

In[19]: # Create a 3x3 array of random integers in the interval [0, 10)
        np.random.randint(0, 10, (3, 3))

Out[19]: array([[2, 3, 4],
                [5, 7, 8],
                [0, 5, 0]])

In[20]: # Create a 3x3 identity matrix
        np.eye(3)

Out[20]: array([[ 1.,  0.,  0.],
                [ 0.,  1.,  0.],
                [ 0.,  0.,  1.]])

In[21]: # Create an uninitialized array of three integers
        # The values will be whatever happens to already exist at that
        # memory location
        np.empty(3)

Out[21]: array([ 1.,  1.,  1.]])
```

NumPy Standard Data Types

- NumPy arrays contain values of a single type, so it is important to have detailed knowledge of those types and their limitations. Because NumPy is built in C, the types will be familiar to users of C, Fortran, and other related languages.

- The standard NumPy data types are listed in Table 2-1. Note that when constructing an array, you can specify them using a string

```
np.zeros(10, dtype='int16')
```

Or using the associated NumPy object:

```
np.zeros(10, dtype=np.int16)
```

Table 2-1. Standard NumPy data types

Data type	Description
bool_	Boolean (True or False) stored as a byte
int_	Default integer type (same as C long; normally either int64 or int32)
intc	Identical to C int (normally int32 or int64)
intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
int8	Byte (–128 to 127)
int16	Integer (–32768 to 32767)
int32	Integer (–2147483648 to 2147483647)
int64	Integer (–9223372036854775808 to 9223372036854775807)
uint8	Unsigned integer (0 to 255)
uint16	Unsigned integer (0 to 65535)
uint32	Unsigned integer (0 to 4294967295)
uint64	Unsigned integer (0 to 18446744073709551615)
float_	Shorthand for float64
float16	Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa
float32	Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa
float64	Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa
complex_	Shorthand for complex128
complex64	Complex number, represented by two 32-bit floats
complex128	Complex number, represented by two 64-bit floats

- More advanced type specification is possible, such as specifying big or little endian numbers; for more information, refer to the NumPy documentation.
- NumPy also supports compound data types, which will be covered in “Structured Data: NumPy’s Structured Arrays”

2. THE BASICS OF NUMPY ARRAYS

- Data manipulation in Python is nearly synonymous with NumPy array manipulation: even newer tools like Pandas are built around the NumPy array.
- This section will present several examples using NumPy array manipulation to access data and subarrays, and to split, reshape, and join the arrays.
- We'll cover a few categories of basic array manipulations here:
 - **Attributes of arrays** - Determining the size, shape, memory consumption, and data types of arrays
 - **Indexing of arrays** - Getting and setting the value of individual array elements
 - **Slicing of arrays** - Getting and setting smaller subarrays within a larger
 - **Reshaping of arrays** - Changing the shape of a given array
 - **Joining and splitting of arrays** - Combining multiple arrays into one, and splitting one array into many

NumPy Array Attributes

- First let's discuss some useful array attributes. We'll start by defining three random arrays: a one-dimensional, two-dimensional, and three-dimensional array.
- We'll use NumPy's random number generator, which we will seed with a set value in order to ensure that the same random arrays are generated each time this code is run:

```
In[1]: import numpy as np
       np.random.seed(0) # seed for reproducibility

       x1 = np.random.randint(10, size=6) # One-dimensional array
       x2 = np.random.randint(10, size=(3, 4)) # Two-dimensional array
       x3 = np.random.randint(10, size=(3, 4, 5)) # Three-dimensional array
```

- Each array has attributes `ndim` (the number of dimensions), `shape` (the size of each dimension), and `size` (the total size of the array):

```
In[2]: print("x3 ndim: ", x3.ndim)
       print("x3 shape:", x3.shape)
       print("x3 size: ", x3.size)
```

```
x3 ndim: 3
x3 shape: (3, 4, 5)
x3 size: 60
```

- Another useful attribute is the `dtype`, the data type of the array

```
In[3]: print("dtype:", x3.dtype)

dtype: int64
```

- Other attributes include `itemsize`, which lists the size (in bytes) of each array element, and `nbytes`, which lists the total size (in bytes) of the array

```
In[4]: print("itemsize:", x3.itemsize, "bytes")
       print("nbytes:", x3.nbytes, "bytes")

itemsize: 8 bytes
nbytes: 480 bytes
```

In general, we expect that `nbytes` is equal to `itemsizes` times `size`

Array Indexing: Accessing Single Elements

- If you are familiar with Python's standard list indexing, indexing in NumPy will feel quite familiar.
- In a one-dimensional array, you can access the *i*th value (counting from zero) by specifying the desired index in square brackets, just as with Python lists:

```
In[5]: x1
Out[5]: array([5, 0, 3, 3, 7, 9])
In[6]: x1[0]
Out[6]: 5
In[7]: x1[4]
Out[7]: 7
```

To index from the end of the array, you can use negative indices:

```
In[8]: x1[-1]
Out[8]: 9
In[9]: x1[-2]
Out[9]: 7
```

In a multidimensional array, you access items using a comma-separated tuple of indices:

```
In[10]: x2
Out[10]: array([[3, 5, 2, 4],
               [7, 6, 8, 8],
               [1, 6, 7, 7]])
In[11]: x2[0, 0]
Out[11]: 3

In[12]: x2[2, 0]
Out[12]: 1
In[13]: x2[2, -1]
Out[13]: 7
```

You can also modify values using any of the above index notation:

```
In[14]: x2[0, 0] = 12
        x2
Out[14]: array([[12, 5, 2, 4],
               [ 7, 6, 8, 8],
               [ 1, 6, 7, 7]])
```

- Keep in mind that, unlike Python lists, NumPy arrays have a fixed type. This means, for example, that if you attempt to insert a floating-point value to an integer array, the value will be silently truncated. Don't be caught unaware by this behavior!

```
In[15]: x1[0] = 3.14159 # this will be truncated!
        x1
```

```
Out[15]: array([3, 0, 3, 3, 7, 9])
```

Array Slicing: Accessing Subarrays

- Just as we can use square brackets to access individual array elements, we can also use them to access subarrays with the slice notation, marked by the colon (:) character.
- The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array *x*, use this:

`x[start : stop : step]`

- If any of these are unspecified, they default to the values `start=0`, `stop=size of dimension`, `step=1`. We'll take a look at accessing subarrays in one dimension and in multiple dimensions.

One-dimensional subarrays

```
In[16]: x = np.arange(10)
        x
```

```
Out[16]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In[17]: x[:5] # first five elements
```

```
Out[17]: array([0, 1, 2, 3, 4])
```

```
In[18]: x[5:] # elements after index 5
```

```
Out[18]: array([5, 6, 7, 8, 9])
```

```
In[19]: x[4:7] # middle subarray
```

```
Out[19]: array([4, 5, 6])
```

```
In[20]: x[::2] # every other element
```

```
Out[20]: array([0, 2, 4, 6, 8])
```

```
In[21]: x[1::2] # every other element, starting at index 1
```

```
Out[21]: array([1, 3, 5, 7, 9])
```

A potentially confusing case is when the step value is negative. In this case, the defaults for `start` and `stop` are swapped. This becomes a convenient way to reverse an array:

```
In[22]: x[::-1] # all elements, reversed
```

```
Out[22]: array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

```
In[23]: x[5::-2] # reversed every other from index 5
```

```
Out[23]: array([5, 3, 1])
```

Multidimensional subarrays

Multidimensional slices work in the same way, with multiple slices separated by commas. For example:

```
In[24]: x2
```

```
Out[24]: array([[12,  5,  2,  4],
                [ 7,  6,  8,  8],
                [ 1,  6,  7,  7]])
```

```
In[25]: x2[:2, :3] # two rows, three columns
```

```
Out[25]: array([[12,  5,  2],
                [ 7,  6,  8]])
```

```
In[26]: x2[:3, ::2] # all rows, every other column
```

```
Out[26]: array([[12,  2],
                [ 7,  8],
                [ 1,  7]])
```

Finally, subarray dimensions can even be reversed together:

```
In[27]: x2[::-1, ::-1]
```

```
Out[27]: array([[ 7,  7,  6,  1],
                [ 8,  8,  6,  7],
                [ 4,  2,  5, 12]])
```

Accessing array rows and columns. One commonly needed routine is accessing single rows or columns of an array. You can do this by combining indexing and slicing, using an empty slice marked by a single colon (:):

```
In[28]: print(x2[:, 0]) # first column of x2
```

```
[12  7  1]
```

```
In[29]: print(x2[0, :]) # first row of x2
```

```
[12  5  2  4]
```

In the case of row access, the empty slice can be omitted for a more compact syntax:

```
In[30]: print(x2[0]) # equivalent to x2[0, :]
```

```
[12  5  2  4]
```

Subarrays as no-copy views

- One important—and extremely useful—thing to know about array slices is that they return views rather than copies of the array data.
- This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies. Consider our two-dimensional array from before:

```
In[31]: print(x2)
```

```
[[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Let's extract a 2x2 subarray from this:

```
In[32]: x2_sub = x2[:2, :2]
        print(x2_sub)
```

```
[[12  5]
 [ 7  6]]
```

Now if we modify this subarray, we'll see that the original array is changed! Observe:

```
In[33]: x2_sub[0, 0] = 99
        print(x2_sub)
```

```
[[99  5]
 [ 7  6]]
```

```
In[34]: print(x2)
```

```
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

This default behavior is actually quite useful: it means that when we work with large datasets, we can access and process pieces of these datasets without the need to copy the underlying data buffer.

Creating copies of arrays

Despite the nice features of array views, it is sometimes useful to instead explicitly copy the data within an array or a subarray. This can be most easily done with the `copy()` method:

```
In[35]: x2_sub_copy = x2[:2, :2].copy()
        print(x2_sub_copy)
```

```
[[99  5]
 [ 7  6]]
```

If we now modify this subarray, the original array is not touched:

```
In[36]: x2_sub_copy[0, 0] = 42
        print(x2_sub_copy)
```

```
[[42  5]
 [ 7  6]]
```

```
In[37]: print(x2)
```

```
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Reshaping of Arrays

- Another useful type of operation is reshaping of arrays. The most flexible way of doing this is with the `reshape()` method.
- For example, if you want to put the numbers 1 through 9 in a 3×3 grid, you can do the following:

```
In[38]: grid = np.arange(1, 10).reshape((3, 3))  
        print(grid)
```

```
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

- Note that for this to work, the size of the initial array must match the size of the reshaped array.
- Where possible, the reshape method will use a no-copy view of the initial array, but with noncontiguous memory buffers this is not always the case.
- Another common reshaping pattern is the conversion of a one-dimensional array into a two-dimensional row or column matrix.
- You can do this with the reshape method, or more easily by making use of the new axis keyword within a slice operation

```
In[39]: x = np.array([1, 2, 3])  
  
        # row vector via reshape  
        x.reshape((1, 3))
```

```
Out[39]: array([[1, 2, 3]])
```

```
In[40]: # row vector via newaxis  
        x[np.newaxis, :]
```

```
Out[40]: array([[1, 2, 3]])
```

```
In[41]: # column vector via reshape  
        x.reshape((3, 1))
```

```
Out[41]: array([[1],  
               [2],  
               [3]])
```

```
In[42]: # column vector via newaxis  
        x[:, np.newaxis]
```

```
Out[42]: array([[1],  
               [2],  
               [3]])
```

Array Concatenation and Splitting

- All of the preceding routines worked on single arrays. It's also possible to combine multiple arrays into one, and to conversely split a single array into multiple arrays. We'll take a look at those operations here.

Concatenation of arrays

- Concatenation, or joining of two arrays in NumPy, is primarily accomplished through the routines `np.concatenate`, `np.vstack`, and `np.hstack`. `np.concatenate` takes a tuple or list of arrays as its first argument, as we can see here:

```
In[43]: x = np.array([1, 2, 3])
        y = np.array([3, 2, 1])
        np.concatenate([x, y])

Out[43]: array([1, 2, 3, 3, 2, 1])
```

You can also concatenate more than two arrays at once:

```
In[44]: z = [99, 99, 99]
        print(np.concatenate([x, y, z]))

[ 1  2  3  3  2  1 99 99 99]
```

`np.concatenate` can also be used for two-dimensional arrays:

```
In[45]: grid = np.array([[1, 2, 3],
                        [4, 5, 6]])

In[46]: # concatenate along the first axis
        np.concatenate([grid, grid])

Out[46]: array([[1, 2, 3],
                [4, 5, 6],
                [1, 2, 3],
                [4, 5, 6]])

In[47]: # concatenate along the second axis (zero-indexed)
        np.concatenate([grid, grid], axis=1)

Out[47]: array([[1, 2, 3, 1, 2, 3],
                [4, 5, 6, 4, 5, 6]])
```

For working with arrays of mixed dimensions, it can be clearer to use the `np.vstack` (vertical stack) and `np.hstack` (horizontal stack) functions:

```
In[48]: x = np.array([1, 2, 3])
        grid = np.array([[9, 8, 7],
                        [6, 5, 4]])

        # vertically stack the arrays
        np.vstack([x, grid])
```

```
Out[48]: array([[1, 2, 3],
               [9, 8, 7],
               [6, 5, 4]])
```

```
In[49]: # horizontally stack the arrays
y = np.array([[99],
              [99]])
np.hstack([grid, y])
```

```
Out[49]: array([[ 9,  8,  7, 99],
               [ 6,  5,  4, 99]])
```

Similarly, `np.dstack` will stack arrays along the third axis.

Splitting of arrays

The opposite of concatenation is splitting, which is implemented by the functions `np.split`, `np.hsplit`, and `np.vsplit`. For each of these, we can pass a list of indices giving the split points:

```
In[50]: x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

```
[1 2 3] [99 99] [3 2 1]
```

Notice that N split points lead to $N + 1$ subarrays. The related functions `np.hsplit` and `np.vsplit` are similar:

```
In[51]: grid = np.arange(16).reshape((4, 4))
grid
```

```
Out[51]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11],
               [12, 13, 14, 15]])
```

```
In[52]: upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]]
```

```
[[ 8  9 10 11]
 [12 13 14 15]]
```

```
In[53]: left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

```
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

- Similarly, `np.dsplit` will split arrays along the third axis.

3. COMPUTATION ON NUMPY ARRAYS: UNIVERSAL FUNCTIONS

- Up until now, we have been discussing some of the basic nuts and bolts of NumPy; in the next few sections, we will dive into the reasons that NumPy is so important in the Python data science world. Namely, it provides an easy and flexible interface to optimized computation with arrays of data.
- Computation on NumPy arrays can be very fast, or it can be very slow.
- The key to making it fast is to use vectorized operations, generally implemented through NumPy's universal functions (ufuncs).
- This section motivates the need for NumPy's ufuncs, which can be used to make repeated calculations on array elements much more efficient.
- It then introduces many of the most common and useful arithmetic ufuncs available in the NumPy package

The Slowness of Loops

- Python's default implementation (known as CPython) does some operations very slowly.
- This is in part due to the dynamic, interpreted nature of the language: the fact that types are flexible, so that sequences of operations cannot be compiled down to efficient machine code as in languages like C and Fortran.
- Recently there have been various attempts to address this weakness: well-known examples are the PyPy project, a just-in-time compiled implementation of Python; the Cython project, which converts Python code to compilable C code; and the Numba project, which converts snippets of Python code to fast LLVM bytecode.
- Each of these has its strengths and weaknesses, but it is safe to say that none of the three approaches has yet surpassed the reach and popularity of the standard CPython engine.
- The relative sluggishness of Python generally manifests itself in situations where many small operations are being repeated—for instance, looping over arrays to operate on each element.
- For example, imagine we have an array of values and we'd like to compute the reciprocal of each. A straightforward approach might look like this

```
In[1]: import numpy as np
        np.random.seed(0)

        def compute_reciprocals(values):
            output = np.empty(len(values))
            for i in range(len(values)):
                output[i] = 1.0 / values[i]
            return output

        values = np.random.randint(1, 10, size=5)
        compute_reciprocals(values)

Out[1]: array([ 0.16666667,  1.          ,  0.25         ,  0.25         ,  0.125        ])
```

- This implementation probably feels fairly natural to someone from, say, a C or Java background. But if we measure the execution time of this code for a large input, we see that this operation is very slow, perhaps surprisingly so! We'll benchmark this with IPython's %timeit magic

```
In[2]: big_array = np.random.randint(1, 100, size=1000000)
       %timeit compute_reciprocals(big_array)
```

1 loop, best of 3: 2.91 s per loop

- It takes several seconds to compute these million operations and to store the result! When even cell phones have processing speeds measured in Giga-FLOPS (i.e., billions of numerical operations per second), this seems almost absurdly slow.
- It turns out that the bottleneck here is not the operations themselves, but the type-checking and function dispatches that CPython must do at each cycle of the loop.
- Each time the reciprocal is computed, Python first examines the object's type and does a dynamic lookup of the correct function to use for that type.
- If we were working in compiled code instead, this type specification would be known before the code executes and the result could be computed much more efficiently.

Introducing UFuncs

- For many types of operations, NumPy provides a convenient interface into just this kind of statically typed, compiled routine. This is known as a vectorized operation.
- You can accomplish this by simply performing an operation on the array, which will then be applied to each element.
- This vectorized approach is designed to push the loop into the compiled layer that underlies NumPy, leading to much faster execution.
- Compare the results of the following two:

```
In[3]: print(compute_reciprocals(values))
       print(1.0 / values)
```

```
[ 0.16666667  1.          0.25         0.25         0.125        ]
[ 0.16666667  1.          0.25         0.25         0.125        ]
```

Looking at the execution time for our big array, we see that it completes orders of magnitude faster than the Python loop:

```
In[4]: %timeit (1.0 / big_array)
```

100 loops, best of 3: 4.6 ms per loop

Vectorized operations in NumPy are implemented via *ufuncs*, whose main purpose is to quickly execute repeated operations on values in NumPy arrays. Ufuncs are extremely flexible—before we saw an operation between a scalar and an array, but we can also operate between two arrays:

```
In[5]: np.arange(5) / np.arange(1, 6)
```

```
Out[5]: array([ 0.          ,  0.5          ,  0.66666667,  0.75          ,  0.8          ])
```

And ufunc operations are not limited to one-dimensional arrays—they can act on multidimensional arrays as well:

```
In[6]: x = np.arange(9).reshape((3, 3))
       2 ** x
```

```
Out[6]: array([[ 1,  2,  4],
               [ 8, 16, 32],
               [64, 128, 256]])
```

- Computations using vectorization through ufuncs are nearly always more efficient than their counterpart implemented through Python loops, especially as the arrays grow in size.
- Any time you see such a loop in a Python script, you should consider whether it can be replaced with a vectorized expression.

Exploring NumPy's UFuncs

- Ufuncs exist in two flavors: unary ufuncs, which operate on a single input, and binary ufuncs, which operate on two inputs.
- We'll see examples of both these types of functions here.

Array arithmetic

- NumPy's ufuncs feel very natural to use because they make use of Python's native arithmetic operators. The standard addition, subtraction, multiplication, and division can all be used:

```
In[7]: x = np.arange(4)
      print("x      =", x)
      print("x + 5 =", x + 5)
      print("x - 5 =", x - 5)
      print("x * 2 =", x * 2)

      print("x / 2 =", x / 2)
      print("x // 2 =", x // 2)  # floor division

x      = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [ 0.  0.5  1.  1.5]
x // 2 = [0 0 1 1]
```

There is also a unary ufunc for negation, a ****** operator for exponentiation, and a **%** operator for modulus:

```
In[8]: print("-x      =", -x)
      print("x ** 2 =", x ** 2)
      print("x % 2  =", x % 2)

-x      = [ 0 -1 -2 -3]
x ** 2 = [0 1 4 9]
x % 2  = [0 1 0 1]
```

In addition, these can be strung together however you wish, and the standard order of operations is respected:

```
In[9]: -(0.5*x + 1) ** 2
Out[9]: array([-1.  , -2.25, -4.  , -6.25])
```

All of these arithmetic operations are simply convenient wrappers around specific functions built into NumPy; for example, the **+** operator is a wrapper for the **add** function:

```
In[10]: np.add(x, 2)
Out[10]: array([2. 3. 4. 5])
```

Table 2-2. Arithmetic operators implemented in NumPy

Operator	Equivalent ufunc	Description
+	<code>np.add</code>	Addition (e.g., $1 + 1 = 2$)
-	<code>np.subtract</code>	Subtraction (e.g., $3 - 2 = 1$)
-	<code>np.negative</code>	Unary negation (e.g., -2)
*	<code>np.multiply</code>	Multiplication (e.g., $2 * 3 = 6$)
/	<code>np.divide</code>	Division (e.g., $3 / 2 = 1.5$)
//	<code>np.floor_divide</code>	Floor division (e.g., $3 // 2 = 1$)
**	<code>np.power</code>	Exponentiation (e.g., $2 ** 3 = 8$)
%	<code>np.mod</code>	Modulus/remainder (e.g., $9 \% 4 = 1$)

Absolute value

Just as NumPy understands Python's built-in arithmetic operators, it also understands Python's built-in absolute value function:

```
In[11]: x = np.array([-2, -1, 0, 1, 2])
        abs(x)
```

```
Out[11]: array([2, 1, 0, 1, 2])
```

The corresponding NumPy ufunc is `np.absolute`, which is also available under the alias `np.abs`:

```
In[12]: np.absolute(x)
```

```
Out[12]: array([2, 1, 0, 1, 2])
```

```
In[13]: np.abs(x)
```

```
Out[13]: array([2, 1, 0, 1, 2])
```

This ufunc can also handle complex data, in which the absolute value returns the magnitude:

```
In[14]: x = np.array([3 - 4j, 4 - 3j, 2 + 0j, 0 + 1j])
        np.abs(x)
```

```
Out[14]: array([ 5.,  5.,  2.,  1.])
```

Trigonometric functions

NumPy provides a large number of useful ufuncs, and some of the most useful for the data scientist are the trigonometric functions. We'll start by defining an array of angles:

```
In[15]: theta = np.linspace(0, np.pi, 3)
```

Now we can compute some trigonometric functions on these values:

```
In[16]: print("theta      = ", theta)
        print("sin(theta) = ", np.sin(theta))
        print("cos(theta) = ", np.cos(theta))
        print("tan(theta) = ", np.tan(theta))

theta      = [ 0.          1.57079633  3.14159265]
sin(theta) = [ 0.00000000e+00  1.00000000e+00  1.22464680e-16]
cos(theta) = [ 1.00000000e+00  6.12323400e-17 -1.00000000e+00]
tan(theta) = [ 0.00000000e+00  1.63312394e+16 -1.22464680e-16]
```

The values are computed to within machine precision, which is why values that should be zero do not always hit exactly zero. Inverse trigonometric functions are also available:

```
In[17]: x = [-1, 0, 1]
        print("x          = ", x)
        print("arcsin(x) = ", np.arcsin(x))
        print("arccos(x) = ", np.arccos(x))
        print("arctan(x) = ", np.arctan(x))

x          = [-1, 0, 1]
arcsin(x) = [-1.57079633  0.          1.57079633]
arccos(x) = [ 3.14159265  1.57079633  0.          ]
arctan(x) = [-0.78539816  0.          0.78539816]
```

Exponents and logarithms

Another common type of operation available in a NumPy ufunc are the exponentials:

```
In[18]: x = [1, 2, 3]
        print("x          = ", x)
        print("e^x        = ", np.exp(x))
        print("2^x        = ", np.exp2(x))
        print("3^x        = ", np.power(3, x))

x          = [1, 2, 3]
e^x        = [ 2.71828183  7.3890561  20.08553692]
2^x        = [ 2.  4.  8.]
3^x        = [ 3  9 27]
```

The inverse of the exponentials, the logarithms, are also available. The basic `np.log` gives the natural logarithm; if you prefer to compute the base-2 logarithm or the base-10 logarithm, these are available as well:

```
In[19]: x = [1, 2, 4, 10]
        print("x          = ", x)
        print("ln(x)       = ", np.log(x))
        print("log2(x)     = ", np.log2(x))
        print("log10(x)    = ", np.log10(x))
```

```
x      = [1, 2, 4, 10]
ln(x)   = [ 0.          0.69314718  1.38629436  2.30258509]
log2(x) = [ 0.          1.          2.          3.32192809]
log10(x) = [ 0.          0.30103    0.60205999  1.          ]
```

There are also some specialized versions that are useful for maintaining precision with very small input:

```
In[20]: x = [0, 0.001, 0.01, 0.1]
        print("exp(x) - 1 =", np.expm1(x))
        print("log(1 + x) =", np.log1p(x))

exp(x) - 1 = [ 0.          0.0010005  0.01005017  0.10517092]
log(1 + x) = [ 0.          0.0009995  0.00995033  0.09531018]
```

When x is very small, these functions give more precise values than if the raw `np.log` or `np.exp` were used.

Specialized ufuncs

- NumPy has many more ufuncs available, including hyperbolic trig functions, bitwise arithmetic, comparison operators, conversions from radians to degrees, rounding and remainders, and much more.
- A look through the NumPy documentation reveals a lot of interesting functionality.
- Another excellent source for more specialized and obscure ufuncs is the submodule `scipy.special`.
- If you want to compute some obscure mathematical function on your data, chances are it is implemented in `scipy.special`.
- There are far too many functions to list them all, but the following snippet shows a couple that might come up in a statistics context

```
In[21]: from scipy import special

In[22]: # Gamma functions (generalized factorials) and related functions
        x = [1, 5, 10]
        print("gamma(x)      =", special.gamma(x))
        print("ln|gamma(x)| =", special.gammaln(x))
        print("beta(x, 2)    =", special.beta(x, 2))

gamma(x)      = [ 1.00000000e+00  2.40000000e+01  3.62880000e+05]
ln|gamma(x)|  = [ 0.          3.17805383  12.80182748]
beta(x, 2)    = [ 0.5          0.03333333  0.00909091]

In[23]: # Error function (integral of Gaussian)
        # its complement, and its inverse
        x = np.array([0, 0.3, 0.7, 1.0])
        print("erf(x)      =", special.erf(x))
        print("erfc(x)     =", special.erfc(x))
        print("erfinv(x)    =", special.erfinv(x))

erf(x)      = [ 0.          0.32862676  0.67780119  0.84270079]
erfc(x)     = [ 1.          0.67137324  0.32219881  0.15729921]
erfinv(x)    = [ 0.          0.27246271  0.73286908  inf]
```

- There are many, many more ufuncs available in both NumPy and scipy.special. Because the documentation of these packages is available online, a web search along the lines of “gamma function python” will generally find the relevant information.

Advanced Ufunc Features

- Many NumPy users make use of ufuncs without ever learning their full set of features.
- We’ll outline a few specialized features of ufuncs here.

Specifying output

- For large calculations, it is sometimes useful to be able to specify the array where the result of the calculation will be stored.
- Rather than creating a temporary array, you can use this to write computation results directly to the memory location where you’d like them to be. For all ufuncs, you can do this using the out argument of the function:

```
In[24]: x = np.arange(5)
        y = np.empty(5)
        np.multiply(x, 10, out=y)
        print(y)

[ 0. 10. 20. 30. 40.]
```

This can even be used with array views. For example, we can write the results of a computation to every other element of a specified array:

```
In[25]: y = np.zeros(10)
        np.power(2, x, out=y[::2])
        print(y)

[ 1.  0.  2.  0.  4.  0.  8.  0. 16.  0.]
```

If we had instead written `y[::2] = 2 ** x`, this would have resulted in the creation of a temporary array to hold the results of `2 ** x`, followed by a second operation copying those values into the `y` array. This doesn’t make much of a difference for such a small computation, but for very large arrays the memory savings from careful use of the out argument can be significant.

Aggregates

For binary ufuncs, there are some interesting aggregates that can be computed directly from the object. For example, if we’d like to *reduce* an array with a particular operation, we can use the `reduce` method of any ufunc. A reduce repeatedly applies a given operation to the elements of an array until only a single result remains.

For example, calling `reduce` on the `add` ufunc returns the sum of all elements in the array:

```
In[26]: x = np.arange(1, 6)
        np.add.reduce(x)

Out[26]: 15
```

Similarly, calling `reduce` on the `multiply` ufunc results in the product of all array elements:

```
In[27]: np.multiply.reduce(x)
```

```
Out[27]: 120
```

If we'd like to store all the intermediate results of the computation, we can instead use `accumulate`:

```
In[28]: np.add.accumulate(x)
```

```
Out[28]: array([ 1,  3,  6, 10, 15])
```

```
In[29]: np.multiply.accumulate(x)
```

```
Out[29]: array([ 1,  2,  6, 24, 120])
```

Note that for these particular cases, there are dedicated NumPy functions to compute the results (`np.sum`, `np.prod`, `np.cumsum`, `np.cumprod`)

Outer products

- Finally, any ufunc can compute the output of all pairs of two different inputs using the outer method. This allows you, in one line, to do things like create a multiplication table

```
In[30]: x = np.arange(1, 6)  
        np.multiply.outer(x, x)
```

```
Out[30]: array([[ 1,  2,  3,  4,  5],  
                [ 2,  4,  6,  8, 10],  
                [ 3,  6,  9, 12, 15],  
                [ 4,  8, 12, 16, 20],  
                [ 5, 10, 15, 20, 25]])
```

- The `ufunc.at` and `ufunc.reduceat` methods, which we'll explore in "Fancy Indexing" are very helpful as well.
- Another extremely useful feature of ufuncs is the ability to operate between arrays of different sizes and shapes, a set of operations known as broadcasting.

4. AGGREGATIONS: MIN, MAX, AND EVERYTHING IN BETWEEN

- Often when you are faced with a large amount of data, a first step is to compute summary statistics for the data in question.
- Perhaps the most common summary statistics are the mean and standard deviation, which allow you to summarize the "typical" values in a dataset, but other aggregates are useful as well (the sum, product, median, minimum and maximum, quantiles, etc.).
- NumPy has fast built-in aggregation functions for working on arrays

Summing the Values in an Array

As a quick example, consider computing the sum of all values in an array. Python itself can do this using the built-in sum function:

```
In[1]: import numpy as np  
  
In[2]: L = np.random.random(100)  
        sum(L)  
  
Out[2]: 55.61209116604941
```

The syntax is quite similar to that of NumPy's sum function, and the result is the same in the simplest case:

```
In[3]: np.sum(L)  
  
Out[3]: 55.612091166049424
```

However, because it executes the operation in compiled code, NumPy's version of the operation is computed much more quickly:

```
In[4]: big_array = np.random.rand(1000000)  
        %timeit sum(big_array)  
        %timeit np.sum(big_array)  
  
10 loops, best of 3: 104 ms per loop  
1000 loops, best of 3: 442 µs per loop
```

Be careful, though: the sum function and the np.sum function are not identical, which can sometimes lead to confusion! In particular, their optional arguments have different meanings, and np.sum is aware of multiple array dimensions, as we will see in the following section.

Minimum and Maximum

Similarly, Python has built-in min and max functions, used to find the minimum value and maximum value of any given array:

```
In[5]: min(big_array), max(big_array)  
  
Out[5]: (1.1717128136634614e-06, 0.9999976784968716)
```

NumPy's corresponding functions have similar syntax, and again operate much more quickly:

```
In[6]: np.min(big_array), np.max(big_array)  
  
Out[6]: (1.1717128136634614e-06, 0.9999976784968716)  
  
In[7]: %timeit min(big_array)  
        %timeit np.min(big_array)  
  
10 loops, best of 3: 82.3 ms per loop  
1000 loops, best of 3: 497 µs per loop
```

For `min`, `max`, `sum`, and several other NumPy aggregates, a shorter syntax is to use methods of the array object itself:

```
In[8]: print(big_array.min(), big_array.max(), big_array.sum())  
1.17171281366e-06 0.999997678497 499911.628197
```

Whenever possible, make sure that you are using the NumPy version of these aggregates when operating on NumPy arrays!

Multidimensional aggregates

One common type of aggregation operation is an aggregate along a row or column. Say you have some data stored in a two-dimensional array:

```
In[9]: M = np.random.random((3, 4))  
       print(M)  
  
[[ 0.8967576  0.03783739  0.75952519  0.06682827]  
 [ 0.8354065  0.99196818  0.19544769  0.43447084]  
 [ 0.66859307  0.15038721  0.37911423  0.6687194 ]]
```

By default, each NumPy aggregation function will return the aggregate over the entire array:

```
In[10]: M.sum()  
Out[10]: 6.0850555667307118
```

Aggregation functions take an additional argument specifying the *axis* along which the aggregate is computed. For example, we can find the minimum value within each column by specifying `axis=0`:

```
In[11]: M.min(axis=0)  
Out[11]: array([ 0.66859307,  0.03783739,  0.19544769,  0.06682827])
```

The function returns four values, corresponding to the four columns of numbers.

Similarly, we can find the maximum value within each row:

```
In[12]: M.max(axis=1)  
Out[12]: array([ 0.8967576 ,  0.99196818,  0.6687194 ])
```

The way the axis is specified here can be confusing to users coming from other languages. The `axis` keyword specifies the *dimension of the array that will be collapsed*, rather than the dimension that will be returned. So specifying `axis=0` means that the

first axis will be collapsed: for two-dimensional arrays, this means that values within each column will be aggregated.

Other aggregation functions

- NumPy provides many other aggregation functions, but we won't discuss them in detail here.
- Additionally, most aggregates have a NaN-safe counterpart that computes the result while ignoring missing values, which are marked by the special IEEE floating-point NaN value.
- Some of these NaN-safe functions were not added until NumPy 1.8, so they will not be available in older NumPy versions.
- Table 2-3 provides a list of useful aggregation functions available in NumPy.

Table 2-3. Aggregation functions available in NumPy

Function Name	NaN-safe Version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute median of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true

Example: What Is the Average Height of US Presidents?

- Aggregates available in NumPy can be extremely useful for summarizing a set of values.
- As a simple example, let's consider the heights of all US presidents.
- This data is available in the file `president_heights.csv`, which is a simple comma-separated list of labels and values:

```
In[13]: !head -4 data/president_heights.csv
```

```
order,name,height(cm)
1,George Washington,189
2,John Adams,170
3,Thomas Jefferson,189
```

- We'll use the Pandas package, which, to read the file and extract this information (note that the heights are measured in centimeters):

```
In[14]: import pandas as pd
        data = pd.read_csv('data/president_heights.csv')
        heights = np.array(data['height(cm)'])
        print(heights)

[189 170 189 163 183 171 185 168 173 183 173 173 175 178 183 193 178 173
 174 183 183 168 170 178 182 180 183 178 182 188 175 179 183 193 182 183
 177 185 188 188 182 185]
```

Now that we have this data array, we can compute a variety of summary statistics:

```
In[15]: print("Mean height:      ", heights.mean())
        print("Standard deviation:", heights.std())
        print("Minimum height:   ", heights.min())
        print("Maximum height:   ", heights.max())

Mean height:      179.738095238
Standard deviation: 6.93184344275
Minimum height:   163
Maximum height:   193
```

Note that in each case, the aggregation operation reduced the entire array to a single summarizing value, which gives us information about the distribution of values. We may also wish to compute quantiles:

```
In[16]: print("25th percentile:  ", np.percentile(heights, 25))
        print("Median:           ", np.median(heights))
        print("75th percentile:  ", np.percentile(heights, 75))

25th percentile:   174.25
Median:           182.0
75th percentile:   183.0
```

We see that the median height of US presidents is 182 cm, or just shy of six feet.

Of course, sometimes it's more useful to see a visual representation of this data, which we can accomplish using tools in Matplotlib (we'll discuss Matplotlib more fully in [Chapter 4](#)). For example, this code generates the chart shown in [Figure 2-3](#):

```
In[17]: %matplotlib inline
        import matplotlib.pyplot as plt
        import seaborn; seaborn.set() # set plot style

In[18]: plt.hist(heights)
        plt.title('Height Distribution of US Presidents')
        plt.xlabel('height (cm)')
        plt.ylabel('number');
```

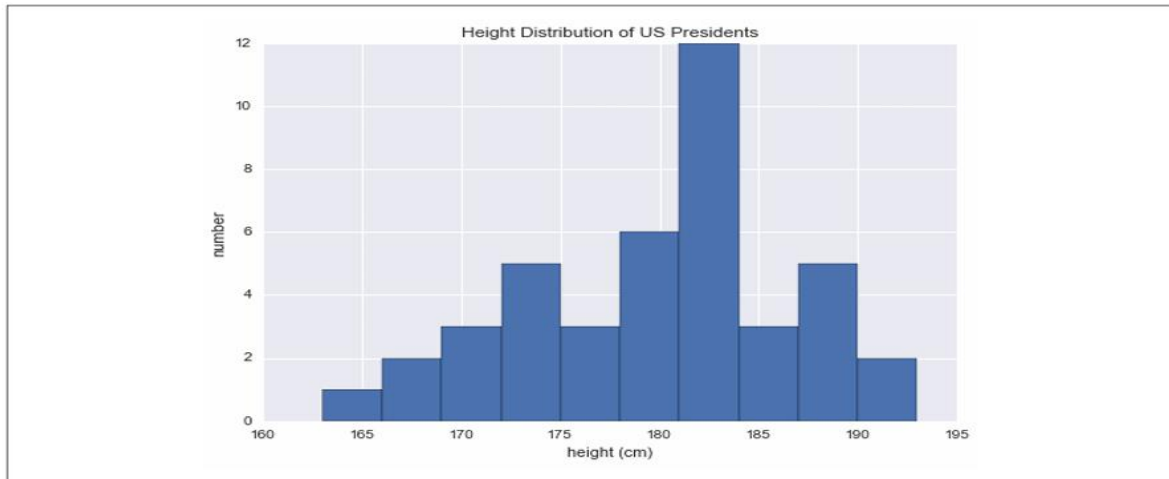


Figure 2-3. Histogram of presidential heights

Data Manipulation with Pandas

- In the previous, we dove into detail on NumPy and its ndarray object, which provides efficient storage and manipulation of dense typed arrays in Python.
- Here we'll build on this knowledge by looking in detail at the data structures provided by the Pandas library.
- Pandas is a newer package built on top of NumPy, and provides an efficient implementation of a DataFrame.
- DataFrames are essentially multidimensional arrays with attached row and column labels, and often with heterogeneous types and/or missing data. As well as offering a convenient storage interface for labeled data, Pandas implements a number of powerful data operations familiar to users of both database frameworks and spreadsheet programs.
- As we saw, NumPy's ndarray data structure provides essential features for the type of clean, well-organized data typically seen in numerical computing tasks.
- While it serves this purpose very well, its limitations become clear when we need more flexibility (attaching labels to data, working with missing data, etc.) and when attempting operations that do not map well to element-wise broadcasting (groupings, pivots, etc.), each of which is an important piece of analyzing the less structured data available in many forms in the world around us.
- Pandas, and in particular its Series and DataFrame objects, builds on the NumPy array structure and provides efficient access to these sorts of "data munging" tasks that occupy much of a data scientist's time.
- In this, we will focus on the mechanics of using Series, DataFrame, and related structures effectively. We will use examples drawn from real datasets where appropriate, but these examples are not necessarily the focus.

INTRODUCING PANDAS OBJECTS

- At the very basic level, Pandas objects can be thought of as enhanced versions of NumPy structured arrays in which the rows and columns are identified with labels rather than simple integer indices.

- As we will see during the course of this chapter, Pandas provides a host of useful tools, methods, and functionality on top of the basic data structures, but nearly everything that follows will require an understanding of what these structures are.
- Thus, before we go any further, let's introduce these three fundamental Pandas data structures: The **Series**, **DataFrame**, and **Index**.
- We will start our code sessions with the standard NumPy and Pandas imports:

```
In[1]: import numpy as np
import pandas as pd
```

The Pandas Series Object

A Pandas **Series** is a one-dimensional array of indexed data. It can be created from a list or array as follows:

```
In[2]: data = pd.Series([0.25, 0.5, 0.75, 1.0])
data

Out[2]: 0    0.25
        1    0.50
        2    0.75
        3    1.00
        dtype: float64
```

As we see in the preceding output, the **Series** wraps both a sequence of values and a sequence of indices, which we can access with the `values` and `index` attributes. The values are simply a familiar NumPy array:

```
In[3]: data.values

Out[3]: array([ 0.25,  0.5 ,  0.75,  1.  ])
```

The `index` is an array-like object of type `pd.Index`, which we'll discuss in more detail momentarily:

```
In[4]: data.index

Out[4]: RangeIndex(start=0, stop=4, step=1)
```

Like with a NumPy array, data can be accessed by the associated index via the familiar Python square-bracket notation:

```
In[5]: data[1]

Out[5]: 0.5

In[6]: data[1:3]

Out[6]: 1    0.50
        2    0.75
        dtype: float64
```

As we will see, though, the Pandas **Series** is much more general and flexible than the one-dimensional NumPy array that it emulates.

Series as generalized NumPy array

- From what we've seen so far, it may look like the Series object is basically interchangeable with a one-dimensional NumPy array.
- The essential difference is the presence of the index: while the NumPy array has an implicitly defined integer index used to access the values, the Pandas Series has an explicitly defined index associated with the values.
- This explicit index definition gives the Series object additional capabilities. For example, the index need not be an integer, but can consist of values of any desired type. For example, if we wish, we can use strings as an index

```
In[7]: data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                        index=['a', 'b', 'c', 'd'])
```

data

```
Out[7]: a    0.25  
       b    0.50  
       c    0.75  
       d    1.00  
       dtype: float64
```

And the item access works as expected:

```
In[8]: data['b']
```

```
Out[8]: 0.5
```

We can even use noncontiguous or nonsequential indices:

```
In[9]: data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                        index=[2, 5, 3, 7])
```

data

```
Out[9]: 2    0.25  
       5    0.50  
       3    0.75  
       7    1.00  
       dtype: float64
```

```
In[10]: data[5]
```

```
Out[10]: 0.5
```

Series as specialized dictionary

- In this way, you can think of a Pandas Series a bit like a specialization of a Python dictionary.
- A dictionary is a structure that maps arbitrary keys to a set of arbitrary values, and a Series is a structure that maps typed keys to a set of typed values.
- This typing is important: just as the type-specific compiled code behind a NumPy array makes it more efficient than a Python list for certain operations, the type information of a Pandas Series makes it much more efficient than Python dictionaries for certain operations.
- We can make the Series-as-dictionary analogy even more clear by constructing a Series object directly from a Python dictionary:

```
In[11]: population_dict = {'California': 38332521,
                           'Texas': 26448193,
                           'New York': 19651127,
                           'Florida': 19552860,
                           'Illinois': 12882135}
population = pd.Series(population_dict)
population
```

```
Out[11]: California    38332521
         Florida      19552860
         Illinois     12882135
         New York     19651127
         Texas        26448193
         dtype: int64
```

By default, a Series will be created where the index is drawn from the sorted keys. From here, typical dictionary-style item access can be performed:

```
In[12]: population['California']
Out[12]: 38332521
```

Unlike a dictionary, though, the Series also supports array-style operations such as slicing:

```
In[13]: population['California':'Illinois']
Out[13]: California    38332521
         Florida      19552860
         Illinois     12882135
         dtype: int64
```

Constructing Series objects

We've already seen a few ways of constructing a Pandas Series from scratch; all of them are some version of the following

```
>>> pd.Series(data, index=index)
```

where `index` is an optional argument, and `data` can be one of many entities.

For example, `data` can be a list or NumPy array, in which case `index` defaults to an integer sequence:

```
In[14]: pd.Series([2, 4, 6])
Out[14]: 0    2
         1    4
         2    6
         dtype: int64
```

data can be a scalar, which is repeated to fill the specified index:

```
In[15]: pd.Series(5, index=[100, 200, 300])
```

```
Out[15]: 100    5  
         200    5  
         300    5  
         dtype: int64
```

data can be a dictionary, in which index defaults to the sorted dictionary keys:

```
In[16]: pd.Series({2:'a', 1:'b', 3:'c'})
```

```
Out[16]: 1    b  
         2    a  
         3    c  
         dtype: object
```

In each case, the index can be explicitly set if a different result is preferred:

```
In[17]: pd.Series({2:'a', 1:'b', 3:'c'}, index=[3, 2])
```

```
Out[17]: 3    c  
         2    a  
         dtype: object
```

Notice that in this case, the Series is populated only with the explicitly identified keys.

The Pandas Dataframe Object

- The next fundamental structure in Pandas is the DataFrame. Like the Series object discussed in the previous section, the DataFrame can be thought of either as a generalization of a NumPy array, or as a specialization of a Python dictionary. We'll now take a look at each of these perspectives.

DataFrame as a generalized NumPy array

- If a Series is an analog of a one-dimensional array with flexible indices, a DataFrame is an analog of a two-dimensional array with both flexible row indices and flexible column names.
- Just as you might think of a two-dimensional array as an ordered sequence of aligned one-dimensional columns, you can think of a DataFrame as a sequence of aligned Series objects. Here, by "aligned" we mean that they share the same index.
- To demonstrate this, let's first construct a new Series listing the area of each of the five states discussed in the previous section:

```
In[18]:  
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,  
            'Florida': 170312, 'Illinois': 149995}
```

```
area = pd.Series(area_dict)
area
```

```
Out[18]: California    423967
         Florida       170312
         Illinois      149995
         New York      141297
         Texas         695662
         dtype: int64
```

Now that we have this along with the population Series from before, we can use a dictionary to construct a single two-dimensional object containing this information:

```
In[19]: states = pd.DataFrame({'population': population,
                              'area': area})
states
```

```
Out[19]:
```

	area	population
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135
New York	141297	19651127
Texas	695662	26448193

Like the Series object, the DataFrame has an `index` attribute that gives access to the index labels:

```
In[20]: states.index
```

```
Out[20]:
Index(['California', 'Florida', 'Illinois', 'New York', 'Texas'], dtype='object')
```

Additionally, the DataFrame has a `columns` attribute, which is an Index object holding the column labels:

```
In[21]: states.columns
```

```
Out[21]: Index(['area', 'population'], dtype='object')
```

Thus the DataFrame can be thought of as a generalization of a two-dimensional NumPy array, where both the rows and columns have a generalized index for accessing the data.

DataFrame as specialized dictionary

Similarly, we can also think of a DataFrame as a specialization of a dictionary. Where a dictionary maps a key to a value, a DataFrame maps a column name to a Series of column data. For example, asking for the `'area'` attribute returns the Series object containing the areas we saw earlier:

```
In[22]: states['area']
```

```
Out[22]: California    423967
         Florida       170312
```

New York 141297
Pennsylvania 119280
Name: area, dtype: int64

- Notice the potential point of confusion here: in a two-dimensional NumPy array, `data[0]` will return the first *row*.
- For a DataFrame, `data['col0']` will return the first *column*. Because of this, it is probably better to think about DataFrames as generalized dictionaries rather than generalized arrays, though both ways of looking at the situation can be useful.
- We'll explore more flexible means of indexing DataFrames in [Data Indexing and Selection](#).

Constructing DataFrame Objects

- A Pandas DataFrame can be constructed in a variety of ways. Here we'll explore several examples.

From a single Series object

- A DataFrame is a collection of Series objects, and a single-column DataFrame can be constructed from a single Series:

```
In [23]: pd.DataFrame(population, columns=['population'])
```

```
Out[23]:
```

	population
California	39538223
Texas	29145505
Florida	21538187
New York	20201249
Pennsylvania	13002700

From a list of dicts

- Any list of dictionaries can be made into a DataFrame. We'll use a simple list comprehension to create some data:

```
In [24]: data = [{'a': i, 'b': 2 * i}
              for i in range(3)]
pd.DataFrame(data)
```

```
Out[24]:
```

	a	b
0	0	0
1	1	2
2	2	4

- Even if some keys in the dictionary are missing, Pandas will fill them in with NaN values (i.e., "Not a Number"; see [Handling Missing Data](#)):

```
In [25]: pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

```
Out[25]:
```

	a	b	c
0	1.0	2	NaN
1	NaN	3	4.0

From a dictionary of Series objects

- As we saw before, a DataFrame can be constructed from a dictionary of Series objects as well:

```
In [26]: pd.DataFrame({'population': population,
                        'area': area})
```

```
Out[26]:
```

	population	area
California	39538223	423967
Texas	29145505	695662
Florida	21538187	170312
New York	20201249	141297
Pennsylvania	13002700	119280

From a two-dimensional NumPy array

- Given a two-dimensional array of data, we can create a DataFrame with any specified column and index names. If omitted, an integer index will be used for each:

```
In [27]: pd.DataFrame(np.random.rand(3, 2),
                        columns=['foo', 'bar'],
                        index=['a', 'b', 'c'])
```

```
Out[27]:
```

	foo	bar
a	0.471098	0.317396
b	0.614766	0.305971
c	0.533596	0.512377

From a NumPy structured array

- We covered structured arrays in [Structured Data: NumPy's Structured Arrays](#). A Pandas DataFrame operates much like a structured array, and can be created directly from one:

```
In [28]: A = np.zeros(3, dtype=[('A', 'i8'), ('B', 'f8')])
A

Out[28]: array([(0, 0.), (0, 0.), (0, 0.)], dtype=[('A', '<i8'), ('B', '<f8')])

In [29]: pd.DataFrame(A)

Out[29]:
```

	A	B
0	0	0.0
1	0	0.0
2	0	0.0

The Pandas Index Object

- As you've seen, the Series and DataFrame objects both contain an explicit *index* that lets you reference and modify data.
- This Index object is an interesting structure in itself, and it can be thought of either as an *immutable array* or as an *ordered set* (technically a multiset, as Index objects may contain repeated values).
- Those views have some interesting consequences in terms of the operations available on Index objects. As a simple example, let's construct an Index from a list of integers:

```
In [30]: ind = pd.Index([2, 3, 5, 7, 11])
ind

Out[30]: Int64Index([2, 3, 5, 7, 11], dtype='int64')
```

Index as Immutable Array

- The Index in many ways operates like an array. For example, we can use standard Python indexing notation to retrieve values or slices:

```
In [31]: ind[1]

Out[31]: 3

In [32]: ind[:2]

Out[32]: Int64Index([2, 5, 11], dtype='int64')
```

Index objects also have many of the attributes familiar from NumPy arrays:

```
In [33]: print(ind.size, ind.shape, ind.ndim, ind.dtype)

5 (5,) 1 int64
```

One difference between **Index** objects and NumPy arrays is that the indices are immutable—that is, they cannot be modified via the normal means:

```
In [34]: ind[1] = 0
```

Index as Ordered Set

- Pandas objects are designed to facilitate operations such as joins across datasets, which depend on many aspects of set arithmetic.
- The Index object follows many of the conventions used by Python's built-in set data structure, so that unions, intersections, differences, and other combinations can be computed in a familiar way:

```
In [35]: indA = pd.Index([1, 3, 5, 7, 9])
         indB = pd.Index([2, 3, 5, 7, 11])
```

```
In [36]: indA.intersection(indB)
```

```
Out[36]: Int64Index([3, 5, 7], dtype='int64')
```

```
In [37]: indA.union(indB)
```

```
Out[37]: Int64Index([1, 2, 3, 5, 7, 9, 11], dtype='int64')
```

```
In [38]: indA.symmetric_difference(indB)
```

```
Out[38]: Int64Index([1, 2, 9, 11], dtype='int64')
```

DATA INDEXING AND SELECTION

- In [Part 2](#), we looked in detail at methods and tools to access, set, and modify values in NumPy arrays.
- These included indexing (e.g., `arr[2, 1]`), slicing (e.g., `arr[:, 1:5]`), masking (e.g., `arr[arr > 0]`), fancy indexing (e.g., `arr[0, [1, 5]]`), and combinations thereof (e.g., `arr[:, [1, 5]]`).
- Here we'll look at similar means of accessing and modifying values in Pandas Series and DataFrame objects. If you have used the NumPy patterns, the corresponding patterns in Pandas will feel very familiar, though there are a few quirks to be aware of.
- We'll start with the simple case of the one-dimensional Series object, and then move on to the more complicated two-dimensional DataFrame object.

Data Selection in Series

- As you saw in the previous chapter, a Series object acts in many ways like a one-dimensional NumPy array, and in many ways like a standard Python dictionary. If you keep these two overlapping analogies in mind, it will help you understand the patterns of data indexing and selection in these arrays.

Series as Dictionary

Like a dictionary, the Series object provides a mapping from a collection of keys to a collection of values:

```
In [1]: import pandas as pd
         data = pd.Series([0.25, 0.5, 0.75, 1.0],
                           index=['a', 'b', 'c', 'd'])
         data
```

```
Out[1]: a    0.25
         b    0.50
         c    0.75
         d    1.00
         dtype: float64
```

```
In [2]: data['b']
```

```
Out[2]: 0.5
```

- We can also use dictionary-like Python expressions and methods to examine the keys/indices and values:

```
In [3]: 'a' in data
```

```
Out[3]: True
```

```
In [4]: data.keys()
```

```
Out[4]: Index(['a', 'b', 'c', 'd'], dtype='object')
```

```
In [5]: list(data.items())
```

```
Out[5]: [('a', 0.25), ('b', 0.5), ('c', 0.75), ('d', 1.0)]
```

- Series objects can also be modified with a dictionary-like syntax. Just as you can extend a dictionary by assigning to a new key, you can extend a Series by assigning to a new index value:

```
In [6]: data['e'] = 1.25
data
```

```
Out[6]: a      0.25
        b      0.50
        c      0.75
        d      1.00
        e      1.25
        dtype: float64
```

- This easy mutability of the objects is a convenient feature: under the hood, Pandas is making decisions about memory layout and data copying that might need to take place, and the user generally does not need to worry about these issues.

Series as One-Dimensional Array

- A Series builds on this dictionary-like interface and provides array-style item selection via the same basic mechanisms as NumPy arrays—that is, slices, masking, and fancy indexing. Examples of these are as follows:

```
In [7]: # slicing by explicit index
data['a':'c']
```

```
Out[7]: a      0.25
        b      0.50
        c      0.75
        dtype: float64
```

```
In [8]: # slicing by implicit integer index
data[0:2]
```

```
Out[8]: a    0.25  
       b    0.50  
       dtype: float64
```

```
In [9]: # masking  
data[(data > 0.3) & (data < 0.8)]
```

```
Out[9]: b    0.50  
       c    0.75  
       dtype: float64
```

```
In [10]: # fancy indexing  
data[['a', 'e']]
```

```
Out[10]: a    0.25  
        e    1.25  
        dtype: float64
```

- Of these, slicing may be the source of the most confusion. Notice that when slicing with an explicit index (e.g., `data['a':'c']`), the final index is *included* in the slice, while when slicing with an implicit index (e.g., `data[0:2]`), the final index is *excluded* from the slice.

Indexers: loc and iloc

- If your Series has an explicit integer index, an indexing operation such as `data[1]` will use the explicit indices, while a slicing operation like `data[1:3]` will use the implicit Python-style indices:

```
In [11]: data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])  
data
```

```
Out[11]: 1    a  
        3    b  
        5    c  
        dtype: object
```

```
In [12]: # explicit index when indexing  
data[1]
```

```
Out[12]: 'a'
```

```
In [13]: # implicit index when slicing  
data[1:3]
```

```
Out[13]: 3    b  
        5    c  
        dtype: object
```

- Because of this potential confusion in the case of integer indexes, Pandas provides some special *indexer* attributes that explicitly expose certain indexing schemes. These are not functional methods, but attributes that expose a particular slicing interface to the data in the Series.
- First, the `loc` attribute allows indexing and slicing that always references the explicit index:

```
In [14]: data.loc[1]
```

```
Out[14]: 'a'
```

```
In [15]: data.loc[1:3]
```

```
Out[15]: 1    a
          3    b
          dtype: object
```

- The `iloc` attribute allows indexing and slicing that always references the implicit Python-style index:

```
In [16]: data.iloc[1]
```

```
Out[16]: 'b'
```

```
In [17]: data.iloc[1:3]
```

```
Out[17]: 3    b
          5    c
          dtype: object
```

- One guiding principle of Python code is that "explicit is better than implicit." The explicit nature of `loc` and `iloc` makes them helpful in maintaining clean and readable code; especially in the case of integer indexes, using them consistently can prevent subtle bugs due to the mixed indexing/slicing convention.

Data Selection in DataFrames

- Recall that a DataFrame acts in many ways like a two-dimensional or structured array, and in other ways like a dictionary of Series structures sharing the same index. These analogies can be helpful to keep in mind as we explore data selection within this structure.

DataFrame as Dictionary

- The first analogy we will consider is the DataFrame as a dictionary of related Series objects. Let's return to our example of areas and populations of states:

```
In [18]: area = pd.Series({'California': 423967, 'Texas': 695662,
                        'Florida': 170312, 'New York': 141297,
                        'Pennsylvania': 119280})
pop = pd.Series({'California': 39538223, 'Texas': 29145505,
                'Florida': 21538187, 'New York': 20201249,
                'Pennsylvania': 13002700})
data = pd.DataFrame({'area':area, 'pop':pop})
data
```

```
Out[18]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187
New York	141297	20201249
Pennsylvania	119280	13002700

- The individual Series that make up the columns of the DataFrame can be accessed via dictionary-style indexing of the column name:

```
In [19]: data['area']
```

```
Out[19]: California    423967
Texas                695662
Florida              170312
New York             141297
Pennsylvania         119280
Name: area, dtype: int64
```

Equivalently, we can use attribute-style access with column names that are strings:

```
In [20]: data.area
```

```
Out[20]: California    423967
Texas                695662
Florida              170312
New York             141297
Pennsylvania         119280
Name: area, dtype: int64
```

- Though this is a useful shorthand, keep in mind that it does not work for all cases! For example, if the column names are not strings, or if the column names conflict with methods of the DataFrame, this attribute-style access is not possible. For example, the DataFrame has a pop method, so data.pop will point to this rather than the pop column:

```
In [21]: data.pop is data["pop"]
```

```
Out[21]: False
```

- In particular, you should avoid the temptation to try column assignment via attributes (i.e., use data['pop'] = z rather than data.pop = z).

- Like with the Series objects discussed earlier, this dictionary-style syntax can also be used to modify the object, in this case adding a new column:

```
In [22]: data['density'] = data['pop'] / data['area']
data
```

```
Out[22]:
```

	area	pop	density
California	423967	39538223	93.257784
Texas	695662	29145505	41.896072
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120
Pennsylvania	119280	13002700	109.009893

- This shows a preview of the straightforward syntax of element-by-element arithmetic between Series objects; we'll dig into this further in Operating on Data in Pandas.

DataFrame as Two-Dimensional Array

- As mentioned previously, we can also view the DataFrame as an enhanced two-dimensional array. We can examine the raw underlying data array using the values attribute:

```
In [23]: data.values
```

```
Out[23]: array([[4.23967000e+05, 3.95382230e+07, 9.32577842e+01],
               [6.95662000e+05, 2.91455050e+07, 4.18960717e+01],
               [1.70312000e+05, 2.15381870e+07, 1.26463121e+02],
               [1.41297000e+05, 2.02012490e+07, 1.42970120e+02],
               [1.19280000e+05, 1.30027000e+07, 1.09009893e+02]])
```

- With this picture in mind, many familiar array-like operations can be done on the DataFrame itself. For example, we can transpose the full DataFrame to swap rows and columns:

```
In [24]: data.T
```

```
Out[24]:
```

	California	Texas	Florida	New York	Pennsylvania
area	4.239670e+05	6.956620e+05	1.703120e+05	1.412970e+05	1.192800e+05
pop	3.953822e+07	2.914550e+07	2.153819e+07	2.020125e+07	1.300270e+07
density	9.325778e+01	4.189607e+01	1.264631e+02	1.429701e+02	1.090099e+02

- When it comes to indexing of a DataFrame object, however, it is clear that the dictionary-style indexing of columns precludes our ability to simply treat it as a NumPy array. In particular, passing a single index to an array accesses a row:

```
In [25]: data.values[0]
```

```
Out[25]: array([4.23967000e+05, 3.95382230e+07, 9.32577842e+01])
```

and passing a single "index" to a `DataFrame` accesses a column:

```
In [26]: data['area']
```

```
Out[26]: California    423967
Texas                695662
Florida              170312
New York             141297
Pennsylvania         119280
Name: area, dtype: int64
```

- Thus, for array-style indexing, we need another convention. Here Pandas again uses the `loc` and `iloc` indexers mentioned earlier.
- Using the `iloc` indexer, we can index the underlying array as if it were a simple NumPy array (using the implicit Python-style index), but the `DataFrame` index and column labels are maintained in the result:

```
In [27]: data.iloc[:3, :2]
```

```
Out[27]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

- Similarly, using the `loc` indexer we can index the underlying data in an array-like style but using the explicit index and column names:

```
In [28]: data.loc[:, 'Florida', : 'pop']
```

```
Out[28]:
```

	area	pop
California	423967	39538223
Texas	695662	29145505
Florida	170312	21538187

- Any of the familiar NumPy-style data access patterns can be used within these indexers. For example, in the `loc` indexer we can combine masking and fancy indexing as follows:

In [29]: `data.loc[data.density > 120, ['pop', 'density']]`

Out[29]:

	pop	density
Florida	21538187	126.463121
New York	20201249	142.970120

- Any of these indexing conventions may also be used to set or modify values; this is done in the standard way that you might be accustomed to from working with NumPy:

In [30]: `data.iloc[0, 2] = 90`
`data`

Out[30]:

	area	pop	density
California	423967	39538223	90.000000
Texas	695662	29145505	41.896072
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120
Pennsylvania	119280	13002700	109.009893

- To build up your fluency in Pandas data manipulation, I suggest spending some time with a simple DataFrame and exploring the types of indexing, slicing, masking, and fancy indexing that are allowed by these various indexing approaches.

Additional Indexing Conventions

- There are a couple of extra indexing conventions that might seem at odds with the preceding discussion, but nevertheless can be useful in practice. First, while *indexing* refers to columns, *slicing* refers to rows:

In [31]: `data['Florida':'New York']`

Out[31]:

	area	pop	density
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120

- Such slices can also refer to rows by number rather than by index:

In [32]: `data[1:3]`

Out[32]:

	area	pop	density
Texas	695662	29145505	41.896072
Florida	170312	21538187	126.463121

- Similarly, direct masking operations are interpreted row-wise rather than column-wise:

```
In [33]: data[data.density > 120]
```

```
Out[33]:
```

	area	pop	density
Florida	170312	21538187	126.463121
New York	141297	20201249	142.970120

- These two conventions are syntactically similar to those on a NumPy array, and while they may not precisely fit the mold of the Pandas conventions, they are included due to their practical utility.

OPERATING ON DATA IN PANDAS

- One of the strengths of NumPy is that it allows us to perform quick element-wise operations, both with basic arithmetic (addition, subtraction, multiplication, etc.) and with more complicated operations (trigonometric functions, exponential and logarithmic functions, etc.).
- Pandas inherits much of this functionality from NumPy, and the ufuncs introduced in Computation on NumPy Arrays: Universal Functions are key to this.
- Pandas includes a couple of useful twists, however: for unary operations like negation and trigonometric functions, these ufuncs will *preserve index and column labels* in the output, and for binary operations such as addition and multiplication, Pandas will automatically *align indices* when passing the objects to the ufunc.
- This means that keeping the context of data and combining data from different sources—both potentially error-prone tasks with raw NumPy arrays—become essentially foolproof with Pandas.
- We will additionally see that there are well-defined operations between one-dimensional Series structures and two-dimensional DataFrame structures.

Ufuncs: Index Preservation

- Because Pandas is designed to work with NumPy, any NumPy ufunc will work on Pandas Series and DataFrame objects. Let's start by defining a simple Series and DataFrame on which to demonstrate this:

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: rng = np.random.default_rng(42)
ser = pd.Series(rng.integers(0, 10, 4))
ser
```

```
Out[2]: 0    0
1    7
2    6
3    4
dtype: int64
```

```
In [3]: df = pd.DataFrame(rng.integers(0, 10, (3, 4)),
                           columns=['A', 'B', 'C', 'D'])
df
```

```
Out[3]:
```

	A	B	C	D
0	4	8	0	6
1	2	0	5	9
2	7	7	7	7

- If we apply a NumPy ufunc on either of these objects, the result will be another Pandas object *with the indices preserved*:

```
In [4]: np.exp(ser)
```

```
Out[4]: 0      1.000000
        1    1096.633158
        2    403.428793
        3     54.598150
        dtype: float64
```

This is true also for more involved sequences of operations

```
In [5]: np.sin(df * np.pi / 4)
```

```
Out[5]:
```

	A	B	C	D
0	1.224647e-16	-2.449294e-16	0.000000	-1.000000
1	1.000000e+00	0.000000e+00	-0.707107	0.707107
2	-7.071068e-01	-7.071068e-01	-0.707107	-0.707107

- Any of the ufuncs discussed in Computation on NumPy Arrays: Universal Functions can be used in a similar manner.

Ufuncs: Index Alignment

- For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation. This is very convenient when working with incomplete data, as we'll see in some of the examples that follow.

Index Alignment in Series

- As an example, suppose we are combining two different data sources and wish to find only the top three US states by *area* and the top three US states by *population*:

```
In [6]: area = pd.Series({'Alaska': 1723337, 'Texas': 695662,
                        'California': 423967}, name='area')
        population = pd.Series({'California': 39538223, 'Texas': 29145505,
                        'Florida': 21538187}, name='population')
```

Let's see what happens when we divide these to compute the population density:

```
In [7]: population / area
```

```
Out[7]: Alaska      NaN
        California  93.257784
        Florida      NaN
        Texas       41.896072
        dtype: float64
```

- The resulting array contains the *union* of indices of the two input arrays, which could be determined directly from these indices

```
In [8]: area.index.union(population.index)
```

```
Out[8]: Index(['Alaska', 'California', 'Florida', 'Texas'], dtype='object')
```

- Any item for which one or the other does not have an entry is marked with NaN, or "Not a Number," which is how Pandas marks missing data (see further discussion of missing data in [Handling Missing Data](#)).
- This index matching is implemented this way for any of Python's built-in arithmetic expressions; any missing values are marked by NaN:

```
In [9]: A = pd.Series([2, 4, 6], index=[0, 1, 2])
        B = pd.Series([1, 3, 5], index=[1, 2, 3])
        A + B
```

```
Out[9]: 0    NaN
        1    5.0
        2    9.0
        3    NaN
        dtype: float64
```

- If using NaN values is not the desired behavior, the fill value can be modified using appropriate object methods in place of the operators.
- For example, calling `A.add(B)` is equivalent to calling `A + B`, but allows optional explicit specification of the fill value for any elements in A or B that might be missing:

```
In [10]: A.add(B, fill_value=0)
```

```
Out[10]: 0    2.0
          1    5.0
          2    9.0
          3    5.0
          dtype: float64
```

Index Alignment in DataFrames

- A similar type of alignment takes place for *both* columns and indices when performing operations on DataFrame objects:

```
In [11]: A = pd.DataFrame(rng.integers(0, 20, (2, 2)),
                          columns=['a', 'b'])
        A
```

```
Out[11]:
```

	a	b
0	10	2
1	16	9

```
In [12]: B = pd.DataFrame(rng.integers(0, 10, (3, 3)),
                        columns=['b', 'a', 'c'])
B
```

```
Out[12]:
```

	b	a	c
0	5	3	1
1	9	7	6
2	4	8	5

```
In [13]: A + B
```

```
Out[13]:
```

	a	b	c
0	13.0	7.0	NaN
1	23.0	18.0	NaN
2	NaN	NaN	NaN

- Notice that indices are aligned correctly irrespective of their order in the two objects, and indices in the result are sorted.
- As was the case with Series, we can use the associated object's arithmetic methods and pass any desired fill_value to be used in place of missing entries. Here we'll fill with the mean of all values in A:

```
In [14]: A.add(B, fill_value=A.values.mean())
```

```
Out[14]:
```

	a	b	c
0	13.00	7.00	10.25
1	23.00	18.00	15.25
2	17.25	13.25	14.25

The following table lists Python operators and their equivalent Pandas object methods:

Python operator	Pandas method(s)
+	add
-	sub , subtract
*	mul , multiply
/	truediv , div , divide
//	floordiv
%	mod
**	pow

Ufuncs: Operations Between DataFrames and Series

- When performing operations between a DataFrame and a Series, the index and column alignment is similarly maintained, and the result is similar to operations between a two-dimensional and one-dimensional NumPy array. Consider one common operation, where we find the difference of a two-dimensional array and one of its rows:

```
In [15]: A = rng.integers(10, size=(3, 4))
A
```

```
Out[15]: array([[4, 4, 2, 0],
               [5, 8, 0, 8],
               [8, 2, 6, 1]])
```

```
In [16]: A - A[0]
```

```
Out[16]: array([[ 0,  0,  0,  0],
               [ 1,  4, -2,  8],
               [ 4, -2,  4,  1]])
```

- According to NumPy's broadcasting rules (see [Computation on Arrays: Broadcasting](#)), subtraction between a two-dimensional array and one of its rows is applied row-wise. In Pandas, the convention similarly operates row-wise by default:

```
In [17]: df = pd.DataFrame(A, columns=['Q', 'R', 'S', 'T'])
df - df.iloc[0]
```

```
Out[17]:
```

	Q	R	S	T
0	0	0	0	0
1	1	4	-2	8
2	4	-2	4	1

- If you would instead like to operate column-wise, you can use the object methods mentioned earlier, while specifying the axis keyword:

```
In [18]: df.subtract(df['R'], axis=0)
```

```
Out[18]:
```

	Q	R	S	T
0	0	0	-2	-4
1	-3	0	-8	0
2	6	0	4	-1

- Note that these DataFrame/Series operations, like the operations discussed previously, will automatically align indices between the two elements:

```
In [19]: halfrow = df.iloc[0, ::2]
         halfrow
```

```
Out[19]: Q      4
         S      2
         Name: 0, dtype: int64
```

```
In [20]: df - halfrow
```

```
Out[20]:
```

	Q	R	S	T
0	0.0	NaN	0.0	NaN
1	1.0	NaN	-2.0	NaN
2	4.0	NaN	4.0	NaN

- This preservation and alignment of indices and columns means that operations on data in Pandas will always maintain the data context, which prevents the common errors that might arise when working with heterogeneous and/or misaligned data in raw NumPy arrays.

HANDLING MISSING DATA

- The difference between data found in many tutorials and data in the real world is that real-world data is rarely clean and homogeneous. In particular, many interesting datasets will have some amount of data missing. To make matters even more complicated, different data sources may indicate missing data in different ways.
- In this chapter, we will discuss some general considerations for missing data, look at how Pandas chooses to represent it, and explore some built-in Pandas tools for handling missing data in Python. Here and throughout the book, I will refer to missing data in general as *null*, *NaN*, or *NA* values.

Trade-offs in Missing Data Conventions

- A number of approaches have been developed to track the presence of missing data in a table or DataFrame. Generally, they revolve around one of two strategies: using a *mask* that globally indicates missing values, or choosing a *sentinel value* that indicates a missing entry.
- In the masking approach, the mask might be an entirely separate Boolean array, or it might involve appropriation of one bit in the data representation to locally indicate the null status of a value.
- In the sentinel approach, the sentinel value could be some data-specific convention, such as indicating a missing integer value with -9999 or some rare bit pattern, or it could be a more global convention, such as indicating a missing floating-point value with NaN (Not a Number), a special value that is part of the IEEE floating-point specification.
- Neither of these approaches is without trade-offs. Use of a separate mask array requires allocation of an additional Boolean array, which adds overhead in both storage and computation. A sentinel value reduces the range of valid values that can be represented, and may require extra (often nonoptimized) logic in CPU and GPU arithmetic, because common special values like NaN are not available for all data types.
- As in most cases where no universally optimal choice exists, different languages and systems use different conventions. For example, the R language uses reserved bit patterns within each data type as sentinel values indicating missing data, while the SciDB system uses an extra byte attached to every cell to indicate an NA state.

Missing Data in Pandas

- The way in which Pandas handles missing values is constrained by its reliance on the NumPy package, which does not have a built-in notion of NA values for non-floating-point data types.
- Perhaps Pandas could have followed R's lead in specifying bit patterns for each individual data type to indicate nullness, but this approach turns out to be rather unwieldy. While R has just 4 main data types, NumPy supports *far* more than this: for example, while R has a single integer type, NumPy supports 14 basic integer types once you account for available bit widths, signedness, and endianness of the encoding. Reserving a specific bit pattern in all available NumPy types would lead to an unwieldy amount of overhead in special-casing various operations for various types, likely even requiring a new fork of the NumPy package. Further, for the smaller data types (such as 8-bit integers), sacrificing a bit to use as a mask would significantly reduce the range of values it can represent.

Because of these constraints and trade-offs, Pandas has two "modes" of storing and manipulating null values:

- The default mode is to use a sentinel-based missing data scheme, with sentinel values NaN or None depending on the type of the data.
- Alternatively, you can opt in to using the nullable data types (dtypes) Pandas provides (discussed later in this chapter), which results in the creation of an accompanying mask array to track missing entries. These missing entries are then presented to the user as the special pd.NA value.

In either case, the data operations and manipulations provided by the Pandas API will handle and propagate those missing entries in a predictable manner. But to develop some intuition into *why* these choices are made, let's dive quickly into the trade-offs inherent in None, NaN, and NA. As usual, we'll start by importing NumPy and Pandas:

```
In [1]: import numpy as np
import pandas as pd
```

None as a Sentinel Value

- For some data types, Pandas uses None as a sentinel value. None is a Python object, which means that any array containing None must have dtype=object—that is, it must be a sequence of Python objects.
- For example, observe what happens if you pass None to a NumPy array:

```
In [2]: vals1 = np.array([1, None, 2, 3])
vals1
```

```
Out[2]: array([1, None, 2, 3], dtype=object)
```

- This dtype=object means that the best common type representation NumPy could infer for the contents of the array is that they are Python objects.
- The downside of using None in this way is that operations on the data will be done at the Python level, with much more overhead than the typically fast operations seen for arrays with native types:

```
In [3]: %timeit np.arange(1E6, dtype=int).sum()
```

```
2.73 ms ± 288 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

```
In [4]: %timeit np.arange(1E6, dtype=object).sum()
```

```
92.1 ms ± 3.42 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

- Further, because Python does not support arithmetic operations with None, aggregations like sum or min will generally lead to an error:

```
In [5]: vals1.sum()
```

```
-----
TypeError                                Traceback (most recent call last)
/var/folders/xs/sptt9bk14s34rgxt7453p03r0000gp/T/ipykernel_91333/1181914653.py in <module>
----> 1 vals1.sum()

~/local/share/venv/python-data-science-handbook-2e-u_kwqDTB/lib/python3.9/site-packages/numpy/core/_methods.py in _sum
m(a, axis, dtype, out, keepdims, initial, where)
    46 def _sum(a, axis=None, dtype=None, out=None, keepdims=False,
    47           initial=_NoValue, where=True):
--> 48     return umr_sum(a, axis, dtype, out, keepdims, initial, where)
    49
    50 def _prod(a, axis=None, dtype=None, out=None, keepdims=False,

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

For this reason, Pandas does not use None as a sentinel in its numerical arrays.

- For this reason, Pandas does not use None as a sentinel in its numerical arrays.

NaN: Missing Numerical Data

- The other missing data sentinel, NaN is different; it is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation:

```
In [6]: vals2 = np.array([1, np.nan, 3, 4])
        vals2
```

```
Out[6]: array([ 1., nan,  3.,  4.])
```

```
In [7]: 1 + np.nan
```

```
Out[7]: nan
```

```
In [8]: 0 * np.nan
```

```
Out[8]: nan
```

- This means that aggregates over the values are well defined (i.e., they don't result in an error) but not always useful

```
In [9]: vals2.sum(), vals2.min(), vals2.max()
```

```
Out[9]: (nan, nan, nan)
```

- That said, NumPy does provide NaN-aware versions of aggregations that will ignore these missing values:

```
In [10]: np.nansum(vals2), np.nanmin(vals2), np.nanmax(vals2)
```

```
Out[10]: (8.0, 1.0, 4.0)
```

- The main downside of NaN is that it is specifically a floating-point value; there is no equivalent NaN value for integers, strings, or other types.

NaN and None in Pandas

- NaN and None both have their place, and Pandas is built to handle the two of them nearly interchangeably, converting between them where appropriate:

```
In [11]: pd.Series([1, np.nan, 2, None])

Out[11]: 0    1.0
         1    NaN
         2    2.0
         3    NaN
         dtype: float64
```

- For types that don't have an available sentinel value, Pandas automatically typecasts when NA values are present. For example, if we set a value in an integer array to np.nan, it will automatically be upcast to a floating-point type to accommodate the NA:

```
In [12]: x = pd.Series(range(2), dtype=int)
         x

Out[12]: 0    0
         1    1
         dtype: int64

In [13]: x[0] = None
         x

Out[13]: 0    NaN
         1    1.0
         dtype: float64
```

- Notice that in addition to casting the integer array to floating point, Pandas automatically converts the None to a NaN value.
- While this type of magic may feel a bit hackish compared to the more unified approach to NA values in domain-specific languages like R, the Pandas sentinel/casting approach works quite well in practice and in my experience only rarely causes issues.
- The following table lists the upcasting conventions in Pandas when NA values are introduced:

Typeclass	Conversion when storing NAs	NA sentinel value
floating	No change	np.nan
object	No change	None or np.nan
integer	Cast to float64	np.nan
boolean	Cast to object	None or np.nan

- Keep in mind that in Pandas, string data is always stored with an object dtype.

Pandas Nullable Dtypes

- In early versions of Pandas, NaN and None as sentinel values were the only missing data representations available. The primary difficulty this introduced was with regard to the

implicit type casting: for example, there was no way to represent a true integer array with missing data.

- To address this difficulty, Pandas later added *nullable dtypes*, which are distinguished from regular dtypes by capitalization of their names (e.g., `pd.Int32` versus `np.int32`). For backward compatibility, these nullable dtypes are only used if specifically requested.
- For example, here is a Series of integers with missing data, created from a list containing all three available markers of missing data:

```
In [14]: pd.Series([1, np.nan, 2, None, pd.NA], dtype='Int32')
```

```
Out[14]: 0      1
         1   <NA>
         2      2
         3   <NA>
         4   <NA>
         dtype: Int32
```

- This representation can be used interchangeably with the others in all the operations explored through the rest of this chapter.

Operating on Null Values

- As we have seen, Pandas treats `None`, `NaN`, and `NA` as essentially interchangeable for indicating missing or null values. To facilitate this convention, Pandas provides several methods for detecting, removing, and replacing null values in Pandas data structures. They are:
 - `isnull`: Generates a Boolean mask indicating missing values
 - `notnull`: Opposite of `isnull`
 - `dropna`: Returns a filtered version of the data
 - `fillna`: Returns a copy of the data with missing values filled or imputed

We will conclude this chapter with a brief exploration and demonstration of these routines.

Detecting Null Values

- Pandas data structures have two useful methods for detecting null data: `isnull` and `notnull`. Either one will return a Boolean mask over the data. For example:

```
In [15]: data = pd.Series([1, np.nan, 'hello', None])
```

```
In [16]: data.isnull()
```

```
Out[16]: 0    False
         1     True
         2    False
         3     True
         dtype: bool
```

- As mentioned in Data Indexing and Selection, Boolean masks can be used directly as a Series or DataFrame index:

```
In [17]: data[data.notnull()]
```

```
Out[17]: 0      1
          2    hello
          dtype: object
```

- The `isnull()` and `notnull()` methods produce similar Boolean results for DataFrame objects.

Dropping Null Values

- In addition to these masking methods, there are the convenience methods `dropna` (which removes NA values) and `fillna` (which fills in NA values). For a Series, the result is straightforward:

```
In [18]: data.dropna()
```

```
Out[18]: 0      1
          2    hello
          dtype: object
```

- For a DataFrame, there are more options. Consider the following DataFrame:

```
In [19]: df = pd.DataFrame([[1,      np.nan, 2],
                             [2,      3,    5],
                             [np.nan, 4,    6]])
df
```

```
Out[19]:
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

- We cannot drop single values from a DataFrame; we can only drop entire rows or columns. Depending on the application, you might want one or the other, so `dropna` includes a number of options for a DataFrame.
- By default, `dropna` will drop all rows in which *any* null value is present:

```
In [20]: df.dropna()
```

```
Out[20]:
```

	0	1	2
1	2.0	3.0	5

- Alternatively, you can drop NA values along a different axis. Using `axis=1` or `axis='columns'` drops all columns containing a null value:

```
In [21]: df.dropna(axis='columns')
```

```
Out[21]:
```

	2
0	2
1	5
2	6

- But this drops some good data as well; you might rather be interested in dropping rows or columns with *all* NA values, or a majority of NA values. This can be specified through the `how` or `thresh` parameters, which allow fine control of the number of nulls to allow through.
- The default is `how='any'`, such that any row or column containing a null value will be dropped. You can also specify `how='all'`, which will only drop rows/columns that contain *all* null values:

```
In [22]: df[3] = np.nan
df
```

```
Out[22]:
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
In [23]: df.dropna(axis='columns', how='all')
```

```
Out[23]:
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

- For finer-grained control, the `thresh` parameter lets you specify a minimum number of non-null values for the row/column to be kept:

```
In [24]: df.dropna(axis='rows', thresh=3)
```

```
Out[24]:
```

	0	1	2	3
1	2.0	3.0	5	NaN

- Here, the first and last rows have been dropped because they each contain only two non-null values.

Filling Null Values

- Sometimes rather than dropping NA values, you'd like to replace them with a valid value. This value might be a single number like zero, or it might be some sort of imputation or interpolation from the good values.
- You could do this in-place using the `isnull` method as a mask, but because it is such a common operation Pandas provides the `fillna` method, which returns a copy of the array with the null values replaced.
- Consider the following Series:

```
In [25]: data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'), data
```

```
Out[25]: a      1
         b    <NA>
         c      2
         d    <NA>
         e      3
         dtype: Int32
```

We can fill NA entries with a single value, such as zero:

```
In [26]: data.fillna(0)
```

```
Out[26]: a      1
         b      0
         c      2
         d      0
         e      3
         dtype: Int32
```

We can specify a forward fill to propagate the previous value forward:

```
In [27]: # forward fill
         data.fillna(method='ffill')
```

```
Out[27]: a      1
         b      1
         c      2
         d      2
         e      3
         dtype: Int32
```

Or we can specify a backward fill to propagate the next values backward:

```
In [28]: # back fill
data.fillna(method='bfill')
```

```
Out[28]: a    1
         b    2
         c    2
         d    3
         e    3
         dtype: Int32
```

In the case of a `DataFrame`, the options are similar, but we can also specify an `axis` along which the fills should take place:

```
In [29]: df
```

```
Out[29]:
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
In [30]: df.fillna(method='ffill', axis=1)
```

```
Out[30]:
```

	0	1	2	3
0	1.0	1.0	2.0	2.0
1	2.0	3.0	5.0	5.0
2	NaN	4.0	6.0	6.0

Notice that if a previous value is not available during a forward fill, the NA value remains.

HIERARCHICAL INDEXING

- Up to this point we've been focused primarily on one-dimensional and two-dimensional data, stored in Pandas Series and DataFrame objects, respectively. Often it is useful to go beyond this and store higher-dimensional data—that is, data indexed by more than one or two keys.
- Early Pandas versions provided Panel and Panel4D objects that could be thought of as 3D or 4D analogs to the 2D DataFrame, but they were somewhat clunky to use in practice. A far more common pattern for handling higher-dimensional data is to make use of *hierarchical indexing* (also known as *multi-indexing*) to incorporate multiple index *levels* within a single index. In this way, higher-dimensional data can be compactly represented within the familiar one-dimensional Series and two-dimensional DataFrame objects. (If you're interested in true *N*-dimensional arrays with Pandas-style flexible indices, you can look into the excellent [Xarray package](#).)
- In this chapter, we'll explore the direct creation of MultiIndex objects; considerations when indexing, slicing, and computing statistics across multiply indexed data; and useful routines for converting between simple and hierarchically indexed representations of data.
- We begin with the standard imports:

```
In [1]: import pandas as pd
import numpy as np
```

A Multiply Indexed Series

Let's start by considering how we might represent two-dimensional data within a one-dimensional Series. For concreteness, we will consider a series of data where each point has a character and numerical key.

The Bad Way

Suppose you would like to track data about states from two different years. Using the Pandas tools we've already covered, you might be tempted to simply use Python tuples as keys:

```
In [2]: index = [('California', 2010), ('California', 2020),  
                ('New York', 2010), ('New York', 2020),  
                ('Texas', 2010), ('Texas', 2020)]  
populations = [37253956, 39538223,  
               19378102, 20201249,  
               25145561, 29145505]  
pop = pd.Series(populations, index=index)  
pop
```

```
Out[2]: (California, 2010)    37253956  
(California, 2020)    39538223  
(New York, 2010)      19378102  
(New York, 2020)      20201249  
(Texas, 2010)         25145561  
(Texas, 2020)         29145505  
dtype: int64
```

With this indexing scheme, you can straightforwardly index or slice the series based on this tuple index:

```
In [3]: pop[('California', 2020):('Texas', 2010)]
```

```
Out[3]: (California, 2020)    39538223  
(New York, 2010)      19378102  
(New York, 2020)      20201249  
(Texas, 2010)         25145561  
dtype: int64
```

But the convenience ends there. For example, if you need to select all values from 2010, you'll need to do some messy (and potentially slow) munging to make it happen:

```
In [4]: pop[[i for i in pop.index if i[1] == 2010]]
```

```
Out[4]: (California, 2010)    37253956  
(New York, 2010)         19378102  
(Texas, 2010)            25145561  
dtype: int64
```

This produces the desired result, but is not as clean (or as efficient for large datasets) as the slicing syntax we've grown to love in Pandas.

The Better Way: The Pandas MultiIndex

Fortunately, Pandas provides a better way. Our tuple-based indexing is essentially a rudimentary multi-index, and the Pandas `MultiIndex` type gives us the types of operations we wish to have. We can create a multi-index from the tuples as follows:

```
In [5]: index = pd.MultiIndex.from_tuples(index)
```

The `MultiIndex` represents multiple *levels* of indexing—in this case, the state names and the years—as well as multiple *labels* for each data point which encode these levels.

If we reindex our series with this `MultiIndex`, we see the hierarchical representation of the data:

```
In [6]: pop = pop.reindex(index)  
pop
```

```
Out[6]: California  2010    37253956  
              2020    39538223  
New York    2010    19378102  
              2020    20201249  
Texas       2010    25145561  
              2020    29145505  
dtype: int64
```

Here the first two columns of the Series representation show the multiple index values, while the third column shows the data. Notice that some entries are missing in the first column: in this multi-index representation, any blank entry indicates the same value as the line above it.

Now to access all data for which the second index is 2020, we can use the Pandas slicing notation:

```
Out[9]: California    2010    37253956  
          2020    39538223  
        New York      2010    19378102  
          2020    20201249  
        Texas         2010    25145561  
          2020    29145505  
dtype: int64
```

- Seeing this, you might wonder why would we bother with hierarchical indexing at all. The reason is simple: just as we were able to use multi-indexing to manipulate two-dimensional data within a one-dimensional Series, we can also use it to manipulate data of three or more dimensions in a Series or DataFrame.
- Each extra level in a multi-index represents an extra dimension of data; taking advantage of this property gives us much more flexibility in the types of data we can represent.
- Concretely, we might want to add another column of demographic data for each state at each year (say, population under 18); with a MultiIndex this is as easy as adding another column to the DataFrame:

```
In [10]: pop_df = pd.DataFrame({'total': pop,
                                'under18': [9284094, 8898092,
                                             4318033, 4181528,
                                             6879014, 7432474]})
pop_df
```

```
Out[10]:
```

		total	under18
California	2010	37253956	9284094
	2020	39538223	8898092
New York	2010	19378102	4318033
	2020	20201249	4181528
Texas	2010	25145561	6879014
	2020	29145505	7432474

In addition, all the ufuncs and other functionality discussed in [Operating on Data in Pandas](#) work with hierarchical indices as well. Here we compute the fraction of people under 18 by year, given the above data:

```
In [11]: f_u18 = pop_df['under18'] / pop_df['total']
f_u18.unstack()
```

```
Out[11]:
```

	2010	2020
California	0.249211	0.225050
New York	0.222831	0.206994
Texas	0.273568	0.255013

This allows us to easily and quickly manipulate and explore even high-dimensional data.

Methods of MultiIndex Creation

The most straightforward way to construct a multiply indexed Series or DataFrame is to simply pass a list of two or more index arrays to the constructor. For example:

```
In [12]: df = pd.DataFrame(np.random.rand(4, 2),
                           index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
                           columns=['data1', 'data2'])
df
```

```
Out[12]:
```

		data1	data2
a	1	0.748464	0.561409
	2	0.379199	0.622461
b	1	0.701679	0.687932
	2	0.436200	0.950664

The work of creating the MultiIndex is done in the background.

Similarly, if you pass a dictionary with appropriate tuples as keys, Pandas will automatically recognize this and use a MultiIndex by default:

```
In [13]: data = {('California', 2010): 37253956,
                  ('California', 2020): 39538223,
                  ('New York', 2010): 19378102,
                  ('New York', 2020): 20201249,
                  ('Texas', 2010): 25145561,
                  ('Texas', 2020): 29145505}
pd.Series(data)
```

```
Out[13]: California 2010 37253956
                  2020 39538223
New York 2010 19378102
          2020 20201249
Texas 2010 25145561
       2020 29145505
dtype: int64
```

Nevertheless, it is sometimes useful to explicitly create a MultiIndex; we'll look at a couple of methods for doing this next.

Explicit MultiIndex Constructors

For more flexibility in how the index is constructed, you can instead use the constructor methods available in the `pd.MultiIndex` class. For example, as we did before, you can construct a MultiIndex from a simple list of arrays giving the index values within each level:

```
In [14]: pd.MultiIndex.from_arrays([['a', 'a', 'b', 'b'], [1, 2, 1, 2]])
```

```
Out[14]: MultiIndex([('a', 1),  
                     ('a', 2),  
                     ('b', 1),  
                     ('b', 2)],  
                    )
```

Or you can construct it from a list of tuples giving the multiple index values of each point:

```
In [15]: pd.MultiIndex.from_tuples([('a', 1), ('a', 2), ('b', 1), ('b', 2)])
```

```
Out[15]: MultiIndex([('a', 1),  
                     ('a', 2),  
                     ('b', 1),  
                     ('b', 2)],  
                    )
```

You can even construct it from a Cartesian product of single indices:

```
In [16]: pd.MultiIndex.from_product([['a', 'b'], [1, 2]])
```

```
Out[16]: MultiIndex([('a', 1),  
                     ('a', 2),  
                     ('b', 1),  
                     ('b', 2)],  
                    )
```

Similarly, you can construct a MultiIndex directly using its internal encoding by passing levels (a list of lists containing available index values for each level) and codes (a list of lists that reference these labels):

```
In [17]: pd.MultiIndex(levels=[['a', 'b'], [1, 2]],  
                       codes=[[0, 0, 1, 1], [0, 1, 0, 1]])
```

```
Out[17]: MultiIndex([('a', 1),  
                     ('a', 2),  
                     ('b', 1),  
                     ('b', 2)],  
                    )
```

Any of these objects can be passed as the index argument when creating a Series or DataFrame, or be passed to the `reindex` method of an existing Series or DataFrame.

MultiIndex Level Names

Sometimes it is convenient to name the levels of the MultiIndex. This can be accomplished by passing the names argument to any of the previously discussed MultiIndex constructors, or by setting the names attribute of the index after the fact:

```
In [18]: pop.index.names = ['state', 'year']
pop
```

```
Out[18]: state      year
California  2010      37253956
           2020      39538223
New York    2010      19378102
           2020      20201249
Texas       2010      25145561
           2020      29145505
dtype: int64
```

With more involved datasets, this can be a useful way to keep track of the meaning of various index values.

MultiIndex for Columns

In a DataFrame, the rows and columns are completely symmetric, and just as the rows can have multiple levels of indices, the columns can have multiple levels as well. Consider the following, which is a mock-up of some (somewhat realistic) medical data:

```
In [19]: # hierarchical indices and columns
index = pd.MultiIndex.from_product([[2013, 2014], [1, 2]],
                                   names=['year', 'visit'])
columns = pd.MultiIndex.from_product([['Bob', 'Guido', 'Sue'], ['HR', 'Temp']],
                                     names=['subject', 'type'])

# mock some data
data = np.round(np.random.randn(4, 6), 1)
data[:, ::2] *= 10
data += 37

# create the DataFrame
health_data = pd.DataFrame(data, index=index, columns=columns)
health_data
```

Out[19]:

	subject	Bob		Guido		Sue	
	type	HR	Temp	HR	Temp	HR	Temp
year	visit						
2013	1	30.0	38.0	56.0	38.3	45.0	35.8
	2	47.0	37.1	27.0	36.0	37.0	36.4
2014	1	51.0	35.9	24.0	36.7	32.0	36.2
	2	49.0	36.3	48.0	39.2	31.0	35.7

This is fundamentally four-dimensional data, where the dimensions are the subject, the measurement type, the year, and the visit number. With this in place we can, for example, index the top-level column by the person's name and get a full Data Frame containing just that person's information:

In [20]: `health_data['Guido']`

Out[20]:

	type	HR	Temp
year	visit		
2013	1	56.0	38.3
	2	27.0	36.0
2014	1	24.0	36.7
	2	48.0	39.2

Indexing and Slicing a MultiIndex

Indexing and slicing on a MultiIndex is designed to be intuitive, and it helps if you think about the indices as added dimensions. We'll first look at indexing multiply indexed Series, and then multiply indexed DataFrame objects.

Multiply Indexed Series

Consider the multiply indexed Series of state populations we saw earlier:

In [21]:

```
pop
```

```
Out[21]: state      year
California 2010      37253956
           2020      39538223
New York   2010      19378102
           2020      20201249
Texas      2010      25145561
           2020      29145505
dtype: int64
```

In [22]:

```
pop['California', 2010]
```

```
Out[22]: 37253956
```

The MultiIndex also supports *partial indexing*, or indexing just one of the levels in the index. The result is another Series, with the lower-level indices maintained:

In [23]:

```
pop['California']
```

```
Out[23]: year
2010      37253956
2020      39538223
dtype: int64
```

Partial slicing is available as well, as long as the MultiIndex is sorted (see the discussion in [Sorted and Unsorted Indices](#)):

In [24]:

```
pop.loc['California':'New York']
```

```
Out[24]: state      year
California 2010      37253956
           2020      39538223
New York   2010      19378102
           2020      20201249
dtype: int64
```

With sorted indices, partial indexing can be performed on lower levels by passing an empty slice in the first index:

```
In [25]: pop[:, 2010]
```

```
Out[25]: state
California    37253956
New York      19378102
Texas         25145561
dtype: int64
```

Other types of indexing and selection (discussed in [Data Indexing and Selection](#)) work as well; for example, selection based on Boolean masks:

```
In [26]: pop[pop > 22000000]
```

```
Out[26]: state  year
California  2010    37253956
           2020    39538223
Texas       2010    25145561
           2020    29145505
dtype: int64
```

Selection based on fancy indexing also works:

```
In [27]: pop[['California', 'Texas']]
```

```
Out[27]: state  year
California  2010    37253956
           2020    39538223
Texas       2010    25145561
           2020    29145505
dtype: int64
```

Multiply Indexed DataFrames

A multiply indexed DataFrame behaves in a similar manner. Consider our toy medical DataFrame from before:

In [28]: `health_data`

Out[28]:

	subject		Bob		Guido		Sue
	type	HR	Temp	HR	Temp	HR	Temp
	year	visit					
2013	1	30.0	38.0	56.0	38.3	45.0	35.8
	2	47.0	37.1	27.0	36.0	37.0	36.4
2014	1	51.0	35.9	24.0	36.7	32.0	36.2
	2	49.0	36.3	48.0	39.2	31.0	35.7

Remember that columns are primary in a DataFrame, and the syntax used for multiply indexed Series applies to the columns. For example, we can recover Guido's heart rate data with a simple operation:

In [29]: `health_data['Guido', 'HR']`

Out[29]:

year	visit	
2013	1	56.0
	2	27.0
2014	1	24.0
	2	48.0

Name: (Guido, HR), dtype: float64

Also, as with the single-index case, we can use the `loc`, `iloc`, and `ix` indexers introduced in [Data Indexing and Selection](#). For example:

In [30]: `health_data.iloc[:2, :2]`

Out[30]:

	subject		Bob
	type	HR	Temp
	year	visit	
2013	1	30.0	38.0
	2	47.0	37.1

These indexers provide an array-like view of the underlying two-dimensional data, but each individual index in loc or iloc can be passed a tuple of multiple indices. For example:

```
In [31]: health_data.loc[:, ('Bob', 'HR')]
```

```
Out[31]: year  visit
          2013  1      30.0
          2013  2      47.0
          2014  1      51.0
          2014  2      49.0
          Name: (Bob, HR), dtype: float64
```

Working with slices within these index tuples is not especially convenient; trying to create a slice within a tuple will lead to a syntax error:

```
In [32]: health_data.loc[:, 1), (:, 'HR')]
```

```
File "/var/folders/xc/sptt9bk14s34rgxt7453p03r0000gp/T/ipykernel_86488/3311942670.py", line 1
    health_data.loc[:, 1), (:, 'HR')]
                      ^
SyntaxError: invalid syntax
```

You could get around this by building the desired slice explicitly using Python's built-in slice function, but a better way in this context is to use an IndexSlice object, which Pandas provides for precisely this situation. For example:

```
In [33]: idx = pd.IndexSlice
          health_data.loc[idx[:, 1], idx[:, 'HR']]
```

```
Out[33]:
```

		subject	Bob	Guido	Sue
		type	HR	HR	HR
year	visit				
2013	1		30.0	56.0	45.0
2014	1		51.0	24.0	32.0

As you can see, there are many ways to interact with data in multiply indexed Series and DataFrames, and as with many tools in this book the best way to become familiar with them is to try them out!

Rearranging Multi-Indexes

One of the keys to working with multiply indexed data is knowing how to effectively transform the data. There are a number of operations that will preserve all the information in the dataset, but rearrange it for the purposes of various computations. We saw a brief example of this in the stack and unstack methods, but there are many more ways to finely control the rearrangement of data between hierarchical indices and columns, and we'll explore them here.

Sorted and Unsorted Indices

Earlier I briefly mentioned a caveat, but I should emphasize it more here. *Many of the MultiIndex slicing operations will fail if the index is not sorted.* Let's take a closer look.

We'll start by creating some simple multiply indexed data where the indices are *not lexographically sorted*:

```
In [34]: index = pd.MultiIndex.from_product(['a', 'c', 'b'], [1, 2])
data = pd.Series(np.random.rand(6), index=index)
data.index.names = ['char', 'int']
data
```

```
Out[34]: char  int
a      1      0.280341
       2      0.097290
c      1      0.206217
       2      0.431771
b      1      0.100183
       2      0.015851
dtype: float64
```

If we try to take a partial slice of this index, it will result in an error:

```
In [35]: try:
          data['a':'b']
except KeyError as e:
    print("KeyError", e)
```

```
KeyError 'Key length (1) was greater than MultiIndex lexsort depth (0)'
```

Although it is not entirely clear from the error message, this is the result of the MultiIndex not being sorted. For various reasons, partial slices and other similar operations require the levels in the MultiIndex to be in sorted (i.e., lexographical) order. Pandas provides a number of convenience routines to perform this type of sorting, such as the `sort_index` and `sortlevel` methods of the DataFrame. We'll use the simplest, `sort_index`, here:

```
In [36]: data = data.sort_index()
data
```

```
Out[36]: char  int
a      1      0.280341
        2      0.097290
b      1      0.100183
        2      0.015851
c      1      0.206217
        2      0.431771
dtype: float64
```

With the index sorted in this way, partial slicing will work as expected:

```
In [37]: data['a':'b']
```

```
Out[37]: char  int
a      1      0.280341
        2      0.097290
b      1      0.100183
        2      0.015851
dtype: float64
```

Stacking and Unstacking Indices

As we saw briefly before, it is possible to convert a dataset from a stacked multi-index to a simple two-dimensional representation, optionally specifying the level to use:

```
In [38]: pop.unstack(level=0)
```

```
Out[38]: state  California  New York  Texas
year
2010  37253956  19378102  25145561
2020  39538223  20201249  29145505
```

```
In [39]: pop.unstack(level=1)
```

```
Out[39]:      year      2010      2020
state
California  37253956  39538223
New York    19378102  20201249
Texas       25145561  29145505
```

The opposite of unstack is stack, which here can be used to recover the original series:

```
In [40]: pop.unstack().stack()
```

```
Out[40]:
```

state	year	
California	2010	37253956
	2020	39538223
New York	2010	19378102
	2020	20201249
Texas	2010	25145561
	2020	29145505

dtype: int64

Index Setting and Resetting

Another way to rearrange hierarchical data is to turn the index labels into columns; this can be accomplished with the `reset_index` method. Calling this on the population dictionary will result in a DataFrame with state and year columns holding the information that was formerly in the index. For clarity, we can optionally specify the name of the data for the column representation:

```
In [41]: pop_flat = pop.reset_index(name='population')
pop_flat
```

```
Out[41]:
```

	state	year	population
0	California	2010	37253956
1	California	2020	39538223
2	New York	2010	19378102
3	New York	2020	20201249
4	Texas	2010	25145561
5	Texas	2020	29145505

A common pattern is to build a MultiIndex from the column values. This can be done with the `set_index` method of the DataFrame, which returns a multiply indexed DataFrame:

```
In [42]: pop_flat.set_index(['state', 'year'])
```

```
Out[42]:
```

	state	year	population
California		2010	37253956
		2020	39538223
New York		2010	19378102
		2020	20201249
Texas		2010	25145561
		2020	29145505

In practice, this type of reindexing is one of the more useful patterns when exploring real-world datasets.

COMBINING DATASETS: CONCAT AND APPEND

```
In[1]: import pandas as pd
import numpy as np
def make_df(cols, ind):
    """Quickly make a DataFrame"""
    data = {c: [str(c) + str(i) for i in ind]
            for c in cols}
    return pd.DataFrame(data, ind)
# example DataFrame
make_df('ABC', range(3))
```

```
Out[2]: A B C
0 A0 B0 C0
1 A1 B1 C1
2 A2 B2 C2
```

```
In[4]: x = [1, 2, 3]
       y = [4, 5, 6]
       z = [7, 8, 9]
       np.concatenate([x, y, z])
```

```
Out[4]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In[5]: x = [[1, 2],
           [3, 4]]
        np.concatenate([x, x], axis=1)
```

```
Out[5]: array([[1, 2, 1, 2],
               [3, 4, 3, 4]])
```

Simple Concatenation with pd.concat

pd.concat(), which has a similar syntax to np.concatenate.

pd.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, copy=True)

pd.concat() can be used for a simple concatenation of Series or DataFrame objects, just as np.concatenate() can be used for simple concatenations of arrays:

```
In[6]:
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
pd.concat([ser1, ser2])
```

```
Out[6]: 1 A
        2 B
        3 C
        4 D
        5 E
        6 F
```

dtype: object

```
In[7]: df1 = make_df('AB', [1, 2])
        df2 = make_df('AB', [3, 4])
        print(df1); print(df2); print(pd.concat([df1, df2]))
```

df1			df2			pd.concat([df1, df2])		
	A	B		A	B		A	B
1	A1	B1	3	A3	B3	1	A1	B1
2	A2	B2	4	A4	B4	2	A2	B2
						3	A3	B3
						4	A4	B4

By default, the concatenation takes place row-wise within the DataFrame (i.e., axis=0). Like np.concatenate, pd.concat allows specification of an axis along which concatenation will take place. We could have equivalently specified axis=1; here we've used the more intuitive axis='col'.

```
In[8]: df3 = make_df('AB', [0, 1])
      df4 = make_df('CD', [0, 1])
      print(df3); print(df4); print(pd.concat([df3, df4], axis='col'))
```

df3			df4			pd.concat([df3, df4], axis='col')				
	A	B		C	D		A	B	C	D
0	A0	B0	0	C0	D0	0	A0	B0	C0	D0
1	A1	B1	1	C1	D1	1	A1	B1	C1	D1

Duplicate indices

Difference between `np.concatenate` and `pd.concat` is that Pandas concatenation *preserves indices*, even if the result will have duplicate indices

```
In[9]: x = make_df('AB', [0, 1])
      y = make_df('AB', [2, 3])
      y.index = x.index # make duplicate indices!
      print(x); print(y); print(pd.concat([x, y]))
```

x			y			pd.concat([x, y])		
	A	B		A	B		A	B
0	A0	B0	0	A2	B2	0	A0	B0
1	A1	B1	1	A3	B3	1	A1	B1
						0	A2	B2
						1	A3	B3

While this is valid within DataFrames, the outcome is often undesirable. `pd.concat()` gives us a few ways to handle it.

Catching the repeats as an error. If you'd like to simply verify that the indices in the result of `pd.concat()` do not overlap, you can specify the `verify_integrity` flag.

With this set to `True`, the concatenation will raise an exception if there are duplicate indices.

The following code is enclosed in "try except block"

```
In[10]: try:
      pd.concat([x, y], verify_integrity=True)
    except ValueError as e:
      print("ValueError:", e)
ValueError: Indexes have overlapping values: [0, 1]
```

Ignoring the index. Sometimes the index itself does not matter, and you would prefer it to simply be ignored. You can specify this option using the `ignore_index` flag. With this set to `True`, the concatenation will create a new integer index for the resulting Series:

```
In[11]: print(x); print(y); print(pd.concat([x, y], ignore_index=True))
```

x			y			pd.concat([x, y], ignore_index=True)		
	A	B		A	B		A	B
0	A0	B0	0	A2	B2	0	A0	B0
1	A1	B1	1	A3	B3	1	A1	B1
						2	A2	B2
						3	A3	B3

Adding MultiIndex keys. Another alternative is to use the keys option to specify a label for the data sources; the result will be a hierarchically indexed series containing the data:

```
In[12]: print(x); print(y); print(pd.concat([x, y], keys=['x', 'y']))
```

x			y			pd.concat([x, y], keys=['x', 'y'])			
	A	B		A	B		A	B	
0	A0	B0	0	A2	B2	x	0	A0	B0
1	A1	B1	1	A3	B3		1	A1	B1
						y	0	A2	B2
							1	A3	B3

concatenation with joins

Concatenating different column datasets.

In the simple examples we just looked at, we were mainly concatenating DataFrames with shared column names. In practice, data from different sources might have different sets of column names, and pd.concat offers several options in this case. Consider the concatenation of the following two DataFrames, which have some (but not all!) columns in common:

```
In[13]: df5 = make_df('ABC', [1, 2])
        df6 = make_df('BCD', [3, 4])
        print(df5); print(df6); print(pd.concat([df5, df6]))
```

df5				df6				pd.concat([df5, df6])				
	A	B	C		B	C	D		A	B	C	D
1	A1	B1	C1	3	B3	C3	D3	1	A1	B1	C1	NaN
2	A2	B2	C2	4	B4	C4	D4	2	A2	B2	C2	NaN
								3	NaN	B3	C3	D3
								4	NaN	B4	C4	D4

By default, the entries for which no data is available are filled with NA values. To change this, we can specify one of several options for the join and join_axes parameters of the concatenate function. By default, the join is a union of the input columns (join='outer'), but we can change this to an intersection of the columns using join='inner':

```
In[14]: print(df5); print(df6);
        print(pd.concat([df5, df6], join='inner'))
```

df5				df6				pd.concat([df5, df6], join='inner')		
	A	B	C		B	C	D		B	C
1	A1	B1	C1	3	B3	C3	D3	1	B1	C1
2	A2	B2	C2	4	B4	C4	D4	2	B2	C2
								3	B3	C3
								4	B4	C4

#Common Columns are added

Another option is to directly specify the index of the remaining columns using the join_axes argument, which takes a list of index objects. Here we'll specify that the returned columns should be the same as those of the first input:

```
df5      A  B  C      df6      B  C  D      pd.concat([df5, df6], join_axes=[df5.columns])
1  A1  B1  C1      3  B3  C3  D3      1  A1  B1  C1
2  A2  B2  C2      4  B4  C4  D4      2  A2  B2  C2
                                     3  NaN  B3  C3
                                     4  NaN  B4  C4
```

Only columns common to df5 are added

The append() method

rather than calling `pd.concat([df1, df2])`, you can simply call `df1.append(df2)`:

```
In[16]: print(df1); print(df2); print(df1.append(df2))
```

```
df1      A  B      df2      A  B      df1.append(df2)
1  A1  B1      3  A3  B3      1  A1  B1
2  A2  B2      4  A4  B4      2  A2  B2
                                     3  A3  B3
                                     4  A4  B4
```

unlike the `append()` and `extend()` methods of Python lists, the `append()` method in Pandas does not modify the original object—instead, it creates a new object with the combined data.

COMBINING DATASETS: MERGE AND JOIN

One essential feature offered by Pandas is its high-performance, in-memory join and merge operations. The main interface for this is the `pd.merge` function.

Relational Algebra

The behaviour implemented in `pd.merge()` is a subset of what is known as *relational algebra*, which is a formal set of rules for manipulating relational data, and forms the conceptual foundation of operations available in most databases

Categories of Joins

The `pd.merge()` function implements a number of types of joins: the *one-to-one*, *many-to-one*, and *many-to-many* joins.

One-to-one joins

```
In[2]:
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],
                    'hire_date': [2004, 2008, 2012, 2014]})
print(df1); print(df2)
```

```
df1      employee      group      df2      employee      hire_date
0      Bob      Accounting      0      Lisa      2004
1      Jake      Engineering      1      Bob      2008
2      Lisa      Engineering      2      Jake      2012
3      Sue      HR      3      Sue      2014
```

```
In[3]: df3 = pd.merge(df1, df2)
df3
```

```
Out[3]:
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

The employee column is common hence the datas are merged. Many-to-one joins

Many-to-one joins are joins in which one of the two key columns contains duplicate entries.

```
In[4]: df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                           'supervisor': ['Carly', 'Guido', 'Steve']})
print(df3); print(df4); print(pd.merge(df3, df4))
```

df3				df4		
	employee	group	hire_date		group	supervisor
0	Bob	Accounting	2008	0	Accounting	Carly
1	Jake	Engineering	2012	1	Engineering	Guido
2	Lisa	Engineering	2004	2	HR	Steve
3	Sue	HR	2014			

```
pd.merge(df3, df4)
```

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

Many-to-many joins

If the key column in both the left and right array contains duplicates, then the result is a many-to-many merge

```
In[5]: df5 = pd.DataFrame({'group': ['Accounting', 'Accounting',
                                       'Engineering', 'Engineering', 'HR', 'HR'],
```

```
'skills': ['math', 'spreadsheets', 'coding', 'linux',  
'spreadsheets', 'organization']}]
```

```
print(df1); print(df5); print(pd.merge(df1, df5))
```

df1			df5		
	employee	group		group	skills
0	Bob	Accounting	0	Accounting	math
1	Jake	Engineering	1	Accounting	spreadsheets
2	Lisa	Engineering	2	Engineering	coding
3	Sue	HR	3	Engineering	linux
			4	HR	spreadsheets
			5	HR	organization

```
pd.merge(df1, df5)
```

	employee	group	skills
0	Bob	Accounting	math
1	Bob	Accounting	spreadsheets
2	Jake	Engineering	coding
3	Jake	Engineering	linux
4	Lisa	Engineering	coding
5	Lisa	Engineering	linux
6	Sue	HR	spreadsheets
7	Sue	HR	organization

Specification of the Merge Key

The on keyword explicitly specify the name of the key column using the on keyword for joining

```
In[6]: print(df1); print(df2); print(pd.merge(df1, df2, on='employee'))
```

df1			df2		
	employee	group		employee	hire_date
0	Bob	Accounting	0	Lisa	2004
1	Jake	Engineering	1	Bob	2008
2	Lisa	Engineering	2	Jake	2012
3	Sue	HR	3	Sue	2014

```
pd.merge(df1, df2, on='employee')
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

The left_on and right_on keywords

At times you may wish to merge two datasets with different column names. we may have a dataset in which the employee name is labeled as “name” rather than “employee”.

```
In[7]:
df3 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                    'salary': [70000, 80000, 120000, 90000]})
print(df1); print(df3);
print(pd.merge(df1, df3, left_on="employee", right_on="name"))
```

df1			df3		
	employee	group		name	salary
0	Bob	Accounting	0	Bob	70000
1	Jake	Engineering	1	Jake	80000
2	Lisa	Engineering	2	Lisa	120000
3	Sue	HR	3	Sue	90000

```
pd.merge(df1, df3, left_on="employee", right_on="name")
```

	employee	group	name	salary
0	Bob	Accounting	Bob	70000
1	Jake	Engineering	Jake	80000
2	Lisa	Engineering	Lisa	120000
3	Sue	HR	Sue	90000

You can drop a redundant column by drop()

```
In[8]:
pd.merge(df1, df3, left_on="employee", right_on="name").drop('name', axis=1)
```

```
Out[8]:
```

	employee	group	salary
0	Bob	Accounting	70000
1	Jake	Engineering	80000
2	Lisa	Engineering	120000
3	Sue	HR	90000

The left_index and right_index keywords

Sometimes, rather than merging on a column, you would instead like to merge on an index.

```
In[9]: df1a = df1.set_index('employee')
df2a = df2.set_index('employee')
print(df1a); print(df2a)
```

df1a		df2a	
	group		hire_date
employee		employee	
Bob	Accounting	Lisa	2004
Jake	Engineering	Bob	2008
Lisa	Engineering	Jake	2012
Sue	HR	Sue	2014

```
In[10]:
print(df1a); print(df2a);
print(pd.merge(df1a, df2a, left_index=True, right_index=True))
```

df1a		df2a	
	group		hire_date
employee		employee	
Bob	Accounting	Lisa	2004
Jake	Engineering	Bob	2008
Lisa	Engineering	Jake	2012
Sue	HR	Sue	2014

```
pd.merge(df1a, df2a, left_index=True, right_index=True)
      group hire_date
```

employee	group	hire_date
Lisa	Engineering	2004
Bob	Accounting	2008
Jake	Engineering	2012
Sue	HR	2014

Index with common names are merged and associated together.

DataFrames implement the `join()` method, which performs a merge that defaults to joining on indices:

```
In[11]: print(df1a); print(df2a); print(df1a.join(df2a))
```

```
df1a.join(df2a)
      group hire_date
```

employee	group	hire_date
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

Specifying Set Arithmetic for Joins

Mary is only merged since its in both df6 and 7

```
In[13]: df6 = pd.DataFrame({'name': ['Peter', 'Paul', 'Mary'],
                             'food': ['fish', 'beans', 'bread']},
                             columns=['name', 'food'])
df7 = pd.DataFrame({'name': ['Mary', 'Joseph'],
                     'drink': ['wine', 'beer']},
                     columns=['name', 'drink'])
print(df6); print(df7); print(pd.merge(df6, df7))
```

df6		df7		pd.merge(df6, df7)		
	name	food		name	drink	
0	Peter	fish	0	Mary	wine	0
1	Paul	beans	1	Joseph	beer	
2	Mary	bread				

Overlapping Column Names: The suffixes Keyword

you may end up in a case where your two input DataFrames have conflicting column names.

```
In[17]: df8 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                             'rank': [1, 2, 3, 4]})
```

```
df9 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                    'rank': [3, 1, 4, 2]})
print(df8); print(df9); print(pd.merge(df8, df9, on="name"))
```

df8			df9			pd.merge(df8, df9, on="name")			
	name	rank		name	rank		name	rank_x	rank_y
0	Bob	1	0	Bob	3	0	Bob	1	3
1	Jake	2	1	Jake	1	1	Jake	2	1
2	Lisa	3	2	Lisa	4	2	Lisa	3	4
3	Sue	4	3	Sue	2	3	Sue	4	2

Aggregation and Grouping

sum(), mean(), median(), min(), and max()

Planets.head() # show from index 0 to 4

Planets Data

```
In[2]: import seaborn as sns
planets = sns.load_dataset('planets')
planets.shape
```

```
Out[2]: (1035, 6)
```

```
In[3]: planets.head()
```

```
Out[3]:
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Simple Aggregation in Pandas In[4]: rng = np.random.RandomState(42)

```
In[7]: df = pd.DataFrame({'A': rng.rand(5),
                          'B': rng.rand(5)})
df
```

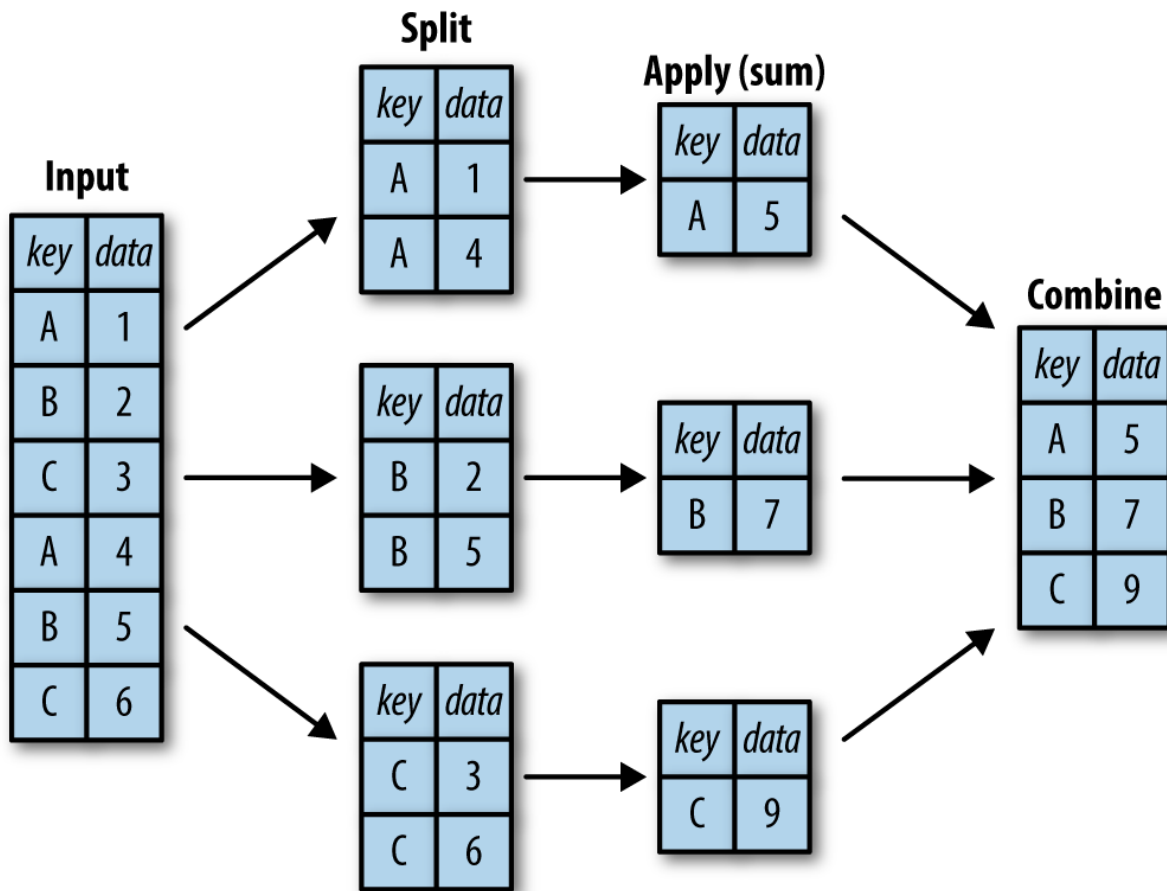
```
Out[7]:
```

	A	B
0	0.155995	0.020584
1	0.058084	0.969910
2	0.866176	0.832443
3	0.601115	0.212339
4	0.708073	0.181825

```
In[8]: df.mean()
```

```
Out[8]: A    0.477888
        B    0.443420
        dtype: float64
```

Group By: Split, Apply, Combine



```
In[11]: df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                           'data': range(6)}, columns=['key', 'data'])
```

```
df
```

```
Out[11]:
```

	key	data
0	A	0
1	B	1
2	C	2
3	A	3
4	B	4
5	C	5

```
In[12]: df.groupby('key')
```

```
Out[12]: <pandas.core.groupby.DataFrameGroupBy object at 0x117272160>
```

```
In[13]: df.groupby('key').sum()
```

```
Out[13]:
```

	key	data
A	3	
B	5	
C	7	

Pivot Tables

Titanic example

```
In[1]: import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
```

```
In[2]: titanic.head()
```

```
In[3]: titanic.groupby('sex')[['survived']].mean()
```

```
Out[3]:      survived
sex
female  0.742038
male    0.188908
```

```
In[4]: titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()
```

```
Out[4]: class      First      Second      Third
sex
female  0.968085  0.921053  0.500000
male    0.368852  0.157407  0.135447
```

Pivot Table Syntax

```
In[5]: titanic.pivot_table('survived', index='sex', columns='class')
```

```
Out[5]: class      First      Second      Third
sex
female  0.968085  0.921053  0.500000
male    0.368852  0.157407  0.135447
```

Multilevel pivot tables

```
In[6]: age = pd.cut(titanic['age'], [0, 18, 80])
titanic.pivot_table('survived', ['sex', age], 'class')
```

```
Out[6]: class      First      Second      Third
sex  age
female (0, 18]  0.909091  1.000000  0.511628
      (18, 80]  0.972973  0.900000  0.423729
male   (0, 18]  0.800000  0.600000  0.215686
      (18, 80]  0.375000  0.071429  0.133663
```

Survival rate from 0 to 18 and 18 to 80